**UNIVERSIDADE PAULISTA – UNIP**

**Projeto Integrado Multidisciplinar**

**Curso Superior de Tecnologia em**

**Análise e Desenvolvimento de Sistemas**

**GABRIEL CARNEIRO SANDES – R661DD5**

**MURILLO APARECIDO CRUZ DOS SANTOS – H74EGJ0**

**RAFAELA ROSATI UNTI – R838BB1**

# TUTORCLASS: PLATAFORMA EDUCACIONAL DE INTERMEDIAÇÃO ENTRE ALUNOS E PROFESSORES PARTICULARES

**Desenvolvimento e implantação de uma solução tecnológica integrada**

São Paulo/SP

2026

**GABRIEL CARNEIRO SANDES – R661DD5**
**MURILLO APARECIDO CRUZ DOS SANTOS – H74EGJ0**
**RAFAELA ROSATI UNTI – R838BB1**

# TUTORCLASS: PLATAFORMA EDUCACIONAL DE INTERMEDIAÇÃO ENTRE ALUNOS E PROFESSORES PARTICULARES

Desenvolvimento e implantação de uma solução tecnológica integrada

Projeto Integrado Multidisciplinar em Análise e Desenvolvimento de Sistemas

Projeto Integrado Multidisciplinar para obtenção do título de tecnólogo em Análise e Desenvolvimento de Sistemas, apresentado à Universidade Paulista – UNIP.

Orientador(a): Prof. Alex Monteiro

São Paulo/SP

2026

## RESUMO

Este trabalho apresenta o desenvolvimento do TutorClass, plataforma educacional de intermediação entre alunos e professores particulares de reforço escolar e de inglês e espanhol, elaborado como Projeto Integrado Multidisciplinar do curso de Análise e Desenvolvimento de Sistemas. O objetivo é documentar uma solução tecnológica integrada, do modelo de negócio à infraestrutura em nuvem. A metodologia combinou pesquisa aplicada, análise da documentação e do código-fonte do projeto e consulta a referências técnicas. O modelo de negócio combina acesso gratuito à contratação de aulas com os planos opcionais Foco e Mentoria, e comissão simulada de 12% sobre as aulas, em valores definidos apenas para simulação acadêmica. São propostas diretrizes de responsabilidade social, diversidade e acessibilidade. A solução Web é uma interface responsiva e acessível (WCAG 2.2, nível AA, com tradução para Libras pelo VLibras), com UX/UI, servida pela própria API REST autenticada por JWT, verificada com testes automatizados e auditoria automática de acessibilidade e executada contra a API real; o aplicativo Mobile, em .NET MAUI, abre essa mesma interface em uma casca nativa Android, conferida em celular físico, e consome a mesma API. A arquitetura do back-end em ASP.NET Core é organizada em camadas, e a persistência utiliza PostgreSQL/Neon, com 22 tabelas, procedures e triggers, e MongoDB Atlas para as mensagens de Mentoria. A API é implantada em contêiner Docker no Render, com versionamento no GitHub, e o planejamento segue o Scrum e o Kanban. Conclui-se que o back-end, a interface Web e a integração com os serviços de dados estão implementados, enquanto a avaliação manual de acessibilidade, a validação com usuários e a validação comercial permanecem como etapas futuras.



**Palavras-chave**: Aulas particulares. Plataforma educacional. API REST. Computação em nuvem. Gerenciamento ágil.

## ABSTRACT

This paper presents the development of TutorClass, an educational platform that connects students and private tutors for school support and English and Spanish classes, prepared as a Multidisciplinary Integrated Project for the Systems Analysis and Development program. The goal is to document an integrated technological solution, from the business model to cloud infrastructure. The methodology combined applied research, analysis of the project documentation and source code, and a review of technical references. The business model combines free access to class booking with the optional Foco and Mentoria subscription plans and a simulated 12% commission on classes, using values defined for academic simulation only. Guidelines for social responsibility, diversity and accessibility are proposed. The Web solution is a responsive and accessible interface (WCAG 2.2, level AA, with Brazilian Sign Language translation through VLibras) with UX/UI, served by the JWT-secured REST API itself, checked with automated tests and an automated accessibility audit, and run against the real API; the Mobile app, built with .NET MAUI, opens this same interface in a native Android shell, checked on a physical phone, and consumes the same API. The ASP.NET Core back-end follows a layered architecture, and persistence relies on PostgreSQL/Neon, with 22 tables, stored procedures and triggers, and on MongoDB Atlas for mentoring messages. The API is deployed in a Docker container on Render, versioned on GitHub, and the planning follows Scrum and Kanban. The back-end, the Web interface and the integration with the data services are implemented, whereas manual accessibility evaluation, validation with users and commercial validation remain as future steps.



**Keywords**: Private tutoring. Educational platform. REST API. Cloud computing. Agile management.

## LISTA DE ABREVIATURAS E SIGLAS

| | |
|---|---|
| **ABNT** | Associação Brasileira de Normas Técnicas |
| **API** | Application Programming Interface |
| **CI/CD** | Continuous Integration / Continuous Delivery |
| **CORS** | Cross-Origin Resource Sharing |
| **DTO** | Data Transfer Object |
| **FAQ** | Frequently Asked Questions |
| **HTTP(S)** | Hypertext Transfer Protocol (Secure) |
| **IA** | Inteligência Artificial |
| **JSON** | JavaScript Object Notation |
| **JWT** | JSON Web Token |
| **MAUI** | Multi-platform App UI |
| **MER** | Modelo Entidade-Relacionamento |
| **MVVM** | Model–View–ViewModel |
| **PIM** | Projeto Integrado Multidisciplinar |
| **REST** | Representational State Transfer |
| **SQL** | Structured Query Language |
| **SUS** | System Usability Scale |
| **TPH** | Table Per Hierarchy |
| **UI** | User Interface |
| **UX** | User Experience |
| **Libras** | Língua Brasileira de Sinais |
| **WCAG** | Web Content Accessibility Guidelines |

**SUMÁRIO**


**1 INTRODUÇÃO**.....6
**1.1 Contextualização e objetivos**.....6
**1.2 Metodologia**.....6
**1.3 Caracterização da organização**.....7
**2 MODELO DE NEGÓCIO E EMPREENDEDORISMO**.....8
**2.1 Problema, oportunidade e objetivo da solução**.....8
**2.2 Público-alvo e participantes**.....8
**2.3 Proposta de valor e diferenciais**.....9
**2.4 Planos e modelo de receita**.....10
**2.5 Operação e recursos necessários**.....10
**2.6 Relacionamento e acompanhamento dos usuários**.....11
**3 RESPONSABILIDADE SOCIAL, DIVERSIDADE E INCLUSÃO**.....12
**3.1 Compromisso social e acesso à educação**.....12
**3.2 Diversidade, respeito e combate à discriminação**.....12
**3.3 Tecnologia inclusiva e acessibilidade digital**.....13
**3.4 Cidadania, proteção dos usuários e limites da verificação**.....14
**4 DESENVOLVIMENTO DA SOLUÇÃO WEB**.....16
**4.1 Arquitetura da aplicação e perfis de usuário**.....16
**4.2 Desenvolvimento web responsivo**.....16
**4.3 APIs REST, autenticação, segurança e integração entre módulos**.....23
**4.4 Estrutura administrativa**.....25
**4.5 UX e UI Design**.....25
**5 DESENVOLVIMENTO DA SOLUÇÃO MOBILE**.....30
**5.1 Objetivo, tecnologias e arquitetura**.....30
**5.2 Telas e navegação**.....30
**5.3 Autenticação, integração e sincronização de dados**.....33
**5.4 Testes e estado da implementação**.....33
**6 ARQUITETURA DE SOFTWARE**.....35
**6.1 Visão geral e organização em camadas**.....35
**6.2 Orientação a objetos e modularização**.....36
**6.3 Fluxo de uma operação, comunicação e segurança**.....36
**7 PROJETO DO BANCO DE DADOS**.....37
**7.1 Modelo lógico e MER relacional (PostgreSQL/Neon)**.....37
**7.2 Modelo documental e integração com o MongoDB Atlas**.....40
**7.3 Procedures, triggers e script consolidado**.....41
**8 INFRAESTRUTURA EM NUVEM E DEVOPS**.....44
**8.1 Arquitetura em nuvem e serviços utilizados**.....44
**8.2 Contêineres, versionamento e fluxo de entrega**.....45
**8.3 Segurança, monitoramento e escalabilidade**.....45
**9 GERENCIAMENTO ÁGIL DO PROJETO**.....47
**9.1 Organização do trabalho e Product Backlog**.....47
**9.2 Sprint Backlog, cronograma e quadro Kanban**.....48
**9.3 Definição de pronto e entregas**.....49
**10 CONCLUSÃO**.....50
**REFERÊNCIAS**.....51

# 1 INTRODUÇÃO

## 1.1 Contextualização e objetivos

A transformação digital tornou-se um dos principais fatores de competitividade das organizações, que buscam soluções capazes de integrar processos, automatizar atividades, ampliar a disponibilidade dos serviços e oferecer melhor experiência aos usuários. No campo educacional, essa tendência se manifesta na procura por plataformas que aproximem estudantes e professores particulares e reduzam o esforço de encontrar, comparar, contratar e acompanhar aulas.

Este Projeto Integrado Multidisciplinar (PIM IV), do curso de Tecnologia em Análise e Desenvolvimento de Sistemas, dá continuidade ao trabalho do PIM III e documenta o desenvolvimento do **TutorClass**, uma plataforma educacional de intermediação entre alunos e professores particulares. O objetivo geral é apresentar uma solução tecnológica integrada, do planejamento do negócio até a infraestrutura em nuvem, passando pelas aplicações Web e Mobile, pela arquitetura de software, pelo banco de dados e pelo gerenciamento ágil do projeto, considerando inovação, inclusão, desempenho, segurança e qualidade.

Como objetivos específicos, o trabalho propõe: (i) delimitar o problema, o público e a proposta de valor da plataforma; (ii) identificar ações de responsabilidade social, diversidade e acessibilidade; (iii) projetar e desenvolver a interface Web acessível e o aplicativo Mobile integrados por uma API REST; (iv) descrever a arquitetura em camadas e a modelagem dos bancos de dados relacional e documental; (v) definir a infraestrutura em nuvem e o fluxo de entrega; e (vi) organizar o planejamento do desenvolvimento segundo princípios ágeis.

## 1.2 Metodologia

O trabalho adota a abordagem de pesquisa aplicada, de natureza descritiva, com desenvolvimento de projeto (GIL, 2017; MARCONI; LAKATOS, 2017). O levantamento reuniu a documentação produzida pelo grupo em cada disciplina do semestre, a análise do código-fonte do repositório do TutorClass (back-end, aplicativo Mobile, testes e script de banco de dados) e a consulta a referências técnicas sobre design responsivo, usabilidade, acessibilidade e gerenciamento ágil. A estrutura segue as etapas propostas no Manual do PIM IV e as normas da ABNT para trabalhos acadêmicos.

Um cuidado metodológico atravessa todo o texto: sempre que possível, distingue-se o que está **implementado e verificável** (por exemplo, o back-end, a API e a persistência) do que é

**proposta de projeto ou validação pendente** (por exemplo, o suporte para iOS e a validação comercial). Os valores financeiros e os repasses apresentados são simulações acadêmicas e não representam transações comerciais reais.

### 1.3 Caracterização da organização

O TutorClass atua no segmento de educação digital, na intermediação de aulas particulares de reforço escolar e de inglês e espanhol. Seu principal serviço é reunir, em um único ambiente, a busca por professores, a consulta a perfis, matérias, disponibilidades e preços, a solicitação e o acompanhamento das aulas. Complementam a oferta os planos opcionais de apoio ao estudo, que incluem o assistente de estudos TutorIA, a Mentoria com chat restrito a vínculos ativos, o acesso a gravações elegíveis e o FAQ/ChatBot de ajuda.

A estrutura organizacional é definida pelos perfis que participam da operação: alunos, responsáveis, professores, administradores e profissionais de suporte. Os principais processos são o cadastro e a análise administrativa das candidaturas e qualificações dos professores; a busca e a solicitação de aulas; a confirmação, a realização e a conclusão da aula; a avaliação e o acompanhamento pós-aula; a contratação de planos e o registro de pagamentos simulados; e a Mentoria.

Os problemas identificados estão na dispersão dessas etapas em canais diferentes, na dificuldade de comparar profissionais, horários e preços e na falta de acompanhamento contínuo após a aula. A oportunidade de melhoria por meio da tecnologia consiste em centralizar o processo em uma plataforma Web e em um aplicativo Mobile conectados a uma mesma API, o que organiza a contratação e permite oferecer recursos de apoio ao estudo conforme o plano de cada aluno.

# 2 MODELO DE NEGÓCIO E EMPREENDEDORISMO

Apresentamos a proposta de negócio do TutorClass, uma plataforma educacional de intermediação entre alunos e professores particulares. O projeto tem como objetivo reunir em um único ambiente digital a busca por professores, a contratação de aulas e o acompanhamento dos serviços e recursos relacionados aos estudos.

Os valores, repasses e demais informações financeiras apresentados nesta seção foram definidos para fins de simulação acadêmica e não representam comprovantes de receitas, transações financeiras ou operações comerciais reais.

## 2.1 Problema, oportunidade e objetivo da solução

A procura por reforço escolar e aulas de inglês e espanhol envolve diferentes etapas, como buscar profissionais, consultar matérias, horários, preços e formas de acompanhamento. O TutorClass propõe reunir essas etapas em um único ambiente digital, permitindo encontrar professores, consultar seus perfis e disponibilidades, solicitar aulas e acompanhar contratações e materiais relacionados às aulas.

A oportunidade de negócio consiste em oferecer uma experiência de contratação mais organizada, juntamente com recursos complementares de apoio ao estudo.

O principal objetivo da solução é conectar professores particulares e alunos por meio de uma plataforma web e um aplicativo mobile integrados a uma API. A solução permite realizar buscas por matéria, consultar perfis e disponibilidades, solicitar aulas particulares, acompanhar o histórico e acessar recursos de acordo com o plano contratado.

As áreas administrativas são responsáveis pelo processo de avaliação das candidaturas e pela qualificação dos professores, contribuindo para o gerenciamento dos profissionais disponibilizados na plataforma.

## 2.2 Público-alvo e participantes

O público-alvo do TutorClass é composto por estudantes que procuram reforço em matérias comuns ou aulas de inglês e espanhol, responsáveis que acompanham ou contratam aulas para alunos vinculados e professores particulares que oferecem seus serviços por meio da plataforma.

Os alunos utilizam a plataforma para buscar professores, contratar aulas, acompanhar seu histórico e acessar os recursos disponíveis de acordo com o plano contratado. Os

responsáveis podem realizar a gestão operacional e financeira das aulas dos alunos vinculados, respeitando as permissões da aplicação. Os professores utilizam a plataforma para divulgar seu perfil e as matérias oferecidas, receber solicitações e, quando habilitados, atuar como mentores.

Administradores e profissionais de suporte participam das operações da plataforma, principalmente nas atividades relacionadas ao gerenciamento dos usuários, análise de candidaturas e acompanhamento dos serviços. Apesar de participarem do funcionamento da solução, não constituem o público comprador principal.

## 2.3 Proposta de valor e diferenciais

A proposta de valor do TutorClass está concentrada na descoberta, contratação e acompanhamento de aulas particulares em um ambiente acessível por diferentes dispositivos. A plataforma busca centralizar as etapas relacionadas à contratação das aulas e oferecer recursos complementares de apoio ao estudo.

Um dos diferenciais definidos no projeto é a verificação da qualificação dos professores. Também fazem parte da proposta os vínculos entre responsáveis e alunos, os recursos de acompanhamento pós-aula, os planos de apoio ao estudo e o chat de Mentoria restrito aos vínculos ativos.

A proposta considera ainda recursos como TutorIA, Mentoria e acesso a gravações elegíveis, conforme o plano contratado e as condições estabelecidas pela plataforma. A Tabela 1 resume os benefícios esperados para cada participante.

**Tabela 1 – Benefícios previstos por participante**

| Participante | Benefício previsto |
|---|---|
| Aluno | Busca de professores por matéria, contratação de aulas, histórico e recursos de estudo conforme o plano. |
| Responsável | Gestão operacional e financeira das aulas de alunos vinculados, respeitando as permissões da aplicação. |
| Professor | Divulgação do perfil e das matérias oferecidas, recebimento de solicitações e possibilidade de atuação como mentor. |
| Plataforma | Receita simulada por assinaturas e por comissão sobre as aulas contratadas. |

Fonte: Elaborado pelos autores (2026).

## 2.4 Planos e modelo de receita

O TutorClass mescla o acesso gratuito à contratação de aulas particulares com assinaturas opcionais. As mensalidades não incluem o preço das aulas, pois a contratação do professor é considerada uma operação separada dentro do modelo proposto. A Tabela 2 apresenta os planos definidos.

**Tabela 2 – Planos, mensalidades e benefícios comerciais previstos**

| Plano | Mensalidade definida | Benefícios comerciais previstos |
|---|---|---|
| Gratuito | R$ 0,00 | Busca e contratação de aulas, FAQ/ChatBot básico, sem TutorIA, Mentoria ou gravações. |
| Foco | R$ 29,99 | Benefícios do plano Gratuito e TutorIA limitada a 7 mensagens diárias. No primeiro mês, mensalidade promocional de R$ 19,99. |
| Mentoria | R$ 79,99 | Benefícios do plano Foco, TutorIA sem limite comercial diário, até 2 mentores ativos, chat e acesso a gravações elegíveis enquanto o plano estiver ativo. No primeiro mês, mensalidade promocional de R$ 59,99. |

Fonte: Elaborado pelos autores (2026). Valores simulados para fins acadêmicos.

Além da receita proveniente das assinaturas, foi definida uma comissão simulada de 12% para a plataforma sobre o valor bruto das aulas contratadas, com repasse previsto de 88% ao professor.

O projeto também considera um adicional simulado de R$ 20,00 por acompanhamento de Mentoria elegível e concluído, limitado a duas ocorrências por par mentor-aluno a cada mês.

Atualmente, o TutorClass não utiliza um gateway de pagamento real. Dessa forma, as cobranças, comissões e repasses apresentados na documentação correspondem às simulações definidas para o projeto acadêmico.

## 2.5 Operação e recursos necessários

A proposta de operação envolve professores cadastrados e submetidos a um processo de análise administrativa, alunos e responsáveis autorizados, além dos serviços tecnológicos necessários para o funcionamento da plataforma.

Após o cadastro e a análise dos professores, os profissionais aprovados podem disponibilizar seus perfis e matérias oferecidas. Alunos e responsáveis podem realizar buscas, consultar as informações dos professores, verificar disponibilidades e solicitar aulas particulares. As contratações e demais informações relacionadas às aulas podem ser acompanhadas por meio do histórico da plataforma.

Os recursos disponíveis dependem do plano contratado e podem incluir TutorIA, Mentoria e gravações elegíveis. As áreas administrativas também participam do processo de gerenciamento e acompanhamento das operações da plataforma.

Para o funcionamento da solução, são utilizados uma plataforma web, um aplicativo mobile, uma API e bancos de dados responsáveis pelo armazenamento das informações. O back-end ASP.NET Core encontra-se organizado em camadas e utiliza PostgreSQL/Neon como banco principal e MongoDB Atlas para as mensagens de Mentoria.

A infraestrutura utilizada no projeto tem como objetivo demonstrar a viabilidade técnica da solução acadêmica. A interface Web é servida pela própria API, na mesma origem; foi verificada com uma API simulada em memória e com testes automatizados e, em seguida, executada pela equipe contra a API real, com a suíte de testes do back-end passando integralmente.

### 2.6 Relacionamento e acompanhamento dos usuários

O relacionamento entre os participantes ocorre por meio dos recursos disponibilizados pela plataforma. Alunos e responsáveis podem acompanhar as informações relacionadas às aulas e utilizar os recursos de apoio disponíveis em seus respectivos planos. Os professores recebem solicitações de aulas e podem utilizar os recursos relacionados ao gerenciamento de seu perfil.

O TutorClass também prevê recursos de comunicação e acompanhamento, como FAQ/ChatBot, TutorIA e chat de Mentoria. O acesso à Mentoria é restrito aos vínculos ativos e está relacionado às condições estabelecidas para o plano contratado.

Dessa forma, o modelo busca manter as informações relacionadas à contratação, às aulas e aos recursos de apoio concentradas no ambiente da plataforma, facilitando o acompanhamento das atividades pelos diferentes participantes.

# 3 RESPONSABILIDADE SOCIAL, DIVERSIDADE E INCLUSÃO

Esta seção apresenta as diretrizes de responsabilidade social, diversidade e acessibilidade previstas para o TutorClass. As medidas descritas como propostas ou requisitos de interface não devem ser confundidas com funcionalidades já implementadas e verificadas, distinção mantida ao longo do texto.

## 3.1 Compromisso social e acesso à educação

O TutorClass busca facilitar o encontro entre estudantes e professores de matérias comuns e dos idiomas inglês e espanhol, organizando busca, contratação e acompanhamento de aulas em um ambiente digital. A existência de um plano gratuito permite pesquisar professores e contratar aulas avulsas; entretanto, as aulas particulares continuam pagas, e a plataforma não deve ser apresentada como um serviço de ensino integralmente gratuito.

Como compromisso social, o projeto prevê comunicação clara sobre preços, limites dos planos e condições de contratação; a oferta de bolsas ou descontos sociais não é afirmada neste trabalho, pois dependeria de decisão e implementação específicas do grupo.

## 3.2 Diversidade, respeito e combate à discriminação

A proposta considera diferentes trajetórias escolares, faixas etárias e formas de aprendizagem, e o atendimento deve respeitar diferenças pessoais e culturais, sem discriminação no cadastro, na busca por professores, no agendamento ou na comunicação entre usuários. Nesse sentido, o TutorClass publica a página **Regras de convivência**, com orientações contra ofensas, assédio, ameaças e tratamento discriminatório, a proteção de crianças e adolescentes e as sanções cabíveis; a página é citada nos Termos de uso, acessível pelo rodapé do site e aberta pela janela de denúncia.

Para fazer valer essas regras, a plataforma implementa denúncia e moderação. Qualquer usuário autenticado, exceto a administração, pode denunciar o perfil de um professor, uma avaliação ou uma mensagem da Mentoria, escolhendo um motivo (conteúdo ofensivo, assédio ou ameaça, discriminação, perfil falso ou golpe, exposição de dados pessoais ou outro) e descrevendo o ocorrido; a identidade do denunciante não é revelada à pessoa denunciada. Para evitar abuso do recurso, o sistema limita o envio a dez denúncias por dia, impede denúncia duplicada enquanto a anterior estiver aberta e não aceita a denúncia contra si mesmo. A administração trata a fila em quatro situações (aberta, em análise, procedente e improcedente),

decide sempre com justificativa e pode aplicar advertência ou suspender a conta, medida reversível por reativação. A suspensão vale de imediato, inclusive para sessões já abertas, porque o acesso de contas inativas é recusado a cada requisição. Os administradores são avisados de cada nova denúncia, o denunciante é avisado da decisão e o usuário advertido recebe um aviso interno.

Permanecem como limitações a ausência de recurso ou contestação da decisão pelo usuário sancionado e de moderação automática de conteúdo; mantém-se, ainda, a verificação administrativa das candidaturas e das qualificações dos professores (seção 4). O mecanismo foi verificado por testes automatizados e pela interface com a API simulada; sua validação com situações reais de uso depende de operação da plataforma.

### 3.3 Tecnologia inclusiva e acessibilidade digital

A interface Web do TutorClass foi desenvolvida com atenção às Diretrizes de Acessibilidade para Conteúdo Web (WCAG) 2.2, nível AA, que organizam o conteúdo em perceptível, operável, compreensível e robusto (W3C, 2023). As diretrizes da Tabela 3 foram implementadas no código do front-end. A conformidade foi verificada com a ferramenta automática axe-core, em 138 combinações de tela, perfil de usuário e estado (computador, largura de 320 px, tema escuro e alto contraste), sem violações detectadas. Em todas as telas, o widget VLibras, do Governo Federal, oferece a tradução do conteúdo para a Língua Brasileira de Sinais (Libras), como apoio adicional às pessoas surdas. Como a verificação automática cobre apenas parte dos critérios, a equipe também testou manualmente o teclado e os atalhos do site, mas leitores de tela e pessoas usuárias de tecnologias assistivas não participaram da avaliação; a menção às WCAG indica orientação de projeto e resultado de verificação automatizada, e não certificação de conformidade.

**Tabela 3 – Diretrizes de acessibilidade e sua implementação na interface Web**

| Diretriz | Aplicação no TutorClass | Implementação no front-end (critério WCAG) |
|---|---|---|
| Navegação por teclado | Garantir que menus, botões, formulários e ações essenciais possam ser acessados sem mouse, com foco visível e ordem de navegação coerente. | Links de salto para o conteúdo e a navegação, anel de foco visível, diálogos nativos com fechamento por Esc, abas por setas e foco levado ao título a cada troca de tela (2.1.1, 2.4.1, 2.4.7). |
| Leitores de tela | Utilizar elementos semânticos, nomes acessíveis para controles e descrições textuais de imagens informativas. | Regiões de referência (cabeçalho, navegação, principal e rodapé), títulos em ordem, tabelas com legenda e regiões *aria-live* para avisos e para a resposta da TutorIA (1.3.1, 4.1.2, 4.1.3). |

| Diretriz | Aplicação no TutorClass | Implementação no front-end (critério WCAG) |
|---|---|---|
| Leitura e compreensão | Usar textos objetivos, rótulos claros e mensagens de erro que expliquem o problema e como corrigi-lo. | Rótulos visíveis, instruções antes dos campos, linguagem simples e avisos temporários de 4 segundos, com pausa ao passar o mouse ou focar e botão para fechar (3.3.2, 2.2.1). |
| Contraste e apresentação | Verificar contraste de texto e controles, tamanho legível de fonte e adaptação da interface à ampliação e a diferentes telas. | Cores com contraste mínimo de 4,5:1, temas claro, escuro e de alto contraste, texto ampliável até 130% e página sem rolagem horizontal a 320 px (1.4.3, 1.4.4, 1.4.10). |
| Formulários e autenticação | Identificar campos obrigatórios e seus erros sem depender exclusivamente de cor; manter instruções de cadastro e login compreensíveis. | Erros ligados aos campos por *aria-describedby*, resumo de erros com links e foco, e indicação além da cor (3.3.1, 3.3.3, 1.4.1). |
| Preferências do usuário | Permitir que a pessoa adapte a interface às suas necessidades. | Painel de acessibilidade com tamanho do texto, tema, alto contraste e redução de movimento, respeitando também as preferências do sistema (1.4.4, 2.3.3). |
| Libras | Oferecer a tradução do texto da página para a Língua Brasileira de Sinais, como apoio às pessoas surdas. | Widget VLibras, do Governo Federal, carregado do serviço oficial depois que a página abre, sem bloquear o carregamento, e exibido pelo botão oficial do VLibras em todas as telas; se o serviço estiver indisponível, o site segue normal (apoio adicional ao critério 1.2.6, de nível AAA). |
| Conteúdos multimídia | Quando houver vídeo ou áudio informativo, avaliar legendas, transcrição e alternativas acessíveis conforme o conteúdo e a viabilidade técnica. | As gravações de aula, simuladas nesta versão, são apresentadas como registros textuais; mídia real exigirá legendas e transcrição (1.2.1, 1.2.2). O vídeo decorativo de fundo das páginas públicas não tem áudio, tem botão para pausar e retomar, começa pausado quando se pede redução de movimento e não aparece nas telas de entrada e nas áreas logadas (2.2.2). |

Fonte: Elaborado pelos autores (2026).

### 3.4 Cidadania, proteção dos usuários e limites da verificação

O sistema contempla os perfis Aluno, Responsável, Professor, Administrador e Suporte. No back-end, os vínculos entre responsável e aluno e as permissões por perfil delimitam quem pode consultar ou executar operações em nome do estudante. A interface explica essas permissões com linguagem simples, para que o acesso financeiro e operacional do responsável não seja confundido com acesso irrestrito a conversas privadas do aluno. Cadastros, pagamentos simulados, histórico de aulas e mensagens de Mentoria só devem ser acessados pelos perfis autorizados, conforme os mecanismos descritos nas seções 4 e 6.

Quanto à proteção de dados pessoais, a interface publica a Política de privacidade e os Termos de uso como páginas do próprio site, acessíveis pelo rodapé e pela página “Privacidade e termos”, com versão e data de vigência. Em todos os cadastros (aluno, responsável e professor), logo abaixo da senha, há uma caixa de aceite obrigatória, cujos links abrem em outra

aba para que a pessoa não perca o que já digitou; sem o aceite, o cadastro não é enviado. Os textos orientam-se pela Lei Geral de Proteção de Dados Pessoais (BRASIL, 2018) e tratam de finalidade, dados coletados, direitos do titular e cuidado redobrado com dados de crianças e adolescentes. Ressalva-se que o aceite é exigido apenas na interface, sem registro de data e versão no banco, que permanece como evolução, e que o canal de contato de privacidade depende de configuração do ambiente de produção. Os textos são um modelo elaborado pela equipe e não substituem revisão jurídica.

A verificação automatizada de contraste, estrutura, nomes acessíveis e refluxo, descrita na seção 3.3, não substitui a avaliação com pessoas. A equipe testou manualmente a navegação por teclado e pelos atalhos do site, e os defeitos encontrados foram corrigidos, mas não houve avaliação com leitores de tela nem com pessoas com deficiência, o que vale também para o aplicativo Mobile, cuja interface é a mesma da versão Web. Por isso, não se afirma que o TutorClass atenda integralmente às WCAG ou ofereça suporte completo a Libras; tais itens permanecem como diretrizes passíveis de priorização no planejamento do projeto.

# 4 DESENVOLVIMENTO DA SOLUÇÃO WEB

Esta seção trata da solução Web do TutorClass em dois eixos complementares: o desenvolvimento web responsivo, com telas principais, adaptação *mobile-first*, integração com a API REST, autenticação, segurança e estrutura administrativa; e o design de experiência e interface (UX/UI), com personas, fluxos, wireframes e análise de usabilidade. O back-end e a API estão implementados, e a interface Web foi implementada como cliente da própria API, verificada com uma API simulada em memória e com testes automatizados de transporte HTTP, roteamento, renderização de texto e acessibilidade; a interface também foi executada pela equipe contra a API real, e a validação com usuários foi feita em caráter exploratório, com cinco pessoas (seção 4.5), e a avaliação com pessoas usuárias de tecnologias assistivas permanece pendente.

## 4.1 Arquitetura da aplicação e perfis de usuário

A aplicação Web segue uma arquitetura cliente-servidor na qual o front-end, construído com HTML, CSS e módulos JavaScript nativos, sem *framework* e sem dependências externas em tempo de execução, é servido pela própria API ASP.NET Core na mesma origem e consome por HTTPS uma API REST em JSON, enquanto o back-end ASP.NET Core concentra regras de negócio, autenticação, autorização e acesso aos dados (MICROSOFT LEARN, [s.d.]). O cliente Web nunca acessa os bancos diretamente. A API já possui política de CORS configurável por origem, com as portas de desenvolvimento usuais de aplicações Web (3000 e 5173) habilitadas somente no ambiente de desenvolvimento.

A interface atende quatro perfis: o **aluno**, que busca professores, solicita aulas e estuda; o **responsável**, que gerencia aulas e finanças dos alunos vinculados; o **professor**, que mantém perfil, matérias, preço por hora e disponibilidade e responde às solicitações; e o **administrador**, que analisa candidaturas e qualificações, mantém o FAQ, modera denúncias e acompanha indicadores. Os objetivos do produto são facilitar a busca por professores por matéria, preço, avaliação e disponibilidade, manter o preço das aulas transparente e funcionar bem em celulares, dispositivo principal de grande parte do público estudantil.

## 4.2 Desenvolvimento web responsivo

O design responsivo combina grades fluidas, imagens flexíveis e *media queries*, de modo que um único site se reorganize em celulares, tablets e desktops (MARCOTTE, 2010). A

abordagem *mobile-first* projeta primeiro para a menor tela e amplia progressivamente o layout, o que obriga a priorizar o essencial (WROBLEWSKI, 2011). Para o TutorClass, as diretrizes adotadas são: CSS base para celular com *media queries* min-width; *meta viewport* configurada; *breakpoints* de partida em 360 px, 768 px, 1024 px e 1280 px, ajustados quando o conteúdo "quebrar"; layout com Flexbox e CSS Grid e unidades relativas; menu inferior ou "hambúrguer" no celular e barra superior no desktop; alvos de toque de no mínimo 44 × 44 px (W3C, 2023); barra de pesquisa e filtros em gaveta no topo da lista, em todos os tamanhos de tela; imagens otimizadas com carregamento tardio; e texto com no mínimo 16 px e bom contraste.

A tela de busca ilustra a adaptação: no topo ficam a barra de pesquisa e o botão "Filtrar e ordenar", que abre a gaveta de filtros; abaixo, os professores aparecem em uma coluna no celular, em duas no tablet e em três no desktop. No celular, o cabeçalho da área pública organiza a marca e o menu em uma linha e os botões Acessibilidade, Entrar e Criar conta em outra, com o mesmo tamanho. A Tabela 4 relaciona as telas principais, suas funções e seus elementos essenciais.

**Tabela 4 – Telas principais da solução Web**

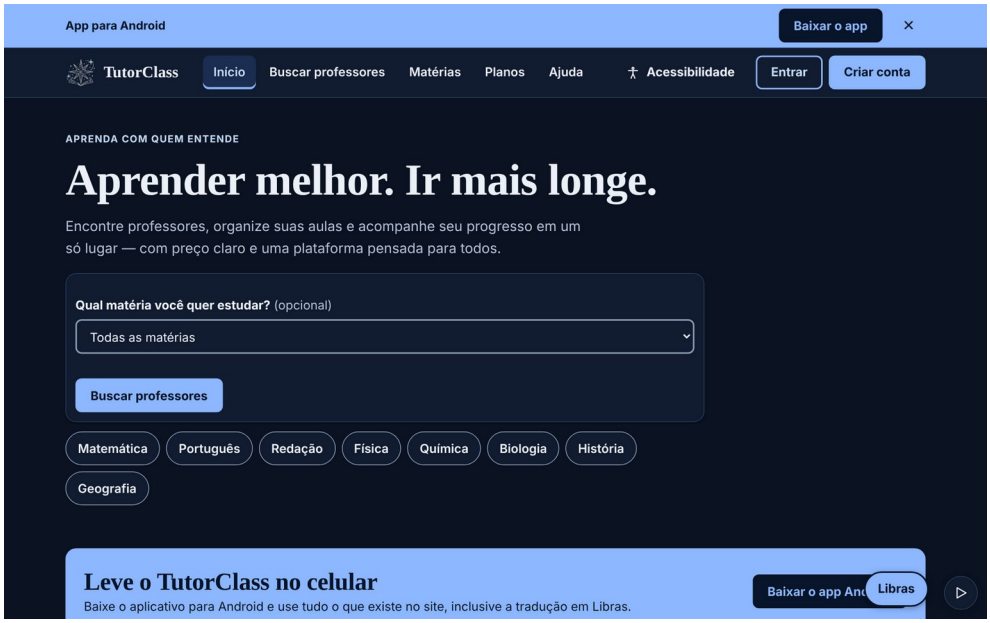
| Tela | Função | Elementos essenciais |
|---|---|---|
| Home | Apresentar a plataforma e iniciar a busca | Busca por matéria, convite para baixar o aplicativo Android, professores em destaque, matérias, chamada para cadastro |
| Cadastro / Login | Criar conta e entrar | Escolha do perfil (aluno, responsável ou professor), e-mail, senha com política de segurança e aceite dos Termos e da Política de privacidade |
| Busca de professores | Listar e filtrar professores | Barra de pesquisa, gaveta de filtros no topo (matéria, preço, nota, disponibilidade), ordenação e cartões de resultado |
| Perfil do professor | Dar confiança e permitir a contratação | Apresentação, matérias, qualificações verificadas, avaliações (com o painel para o aluno avaliar após uma aula concluída), preço por hora, agenda e botão "Solicitar aula" |
| Solicitação e agendamento | Escolher horário e confirmar | Calendário com horários livres, duração, preço calculado pela plataforma e confirmação |
| Painel do aluno e do responsável | Acompanhar a rotina | Visão geral, barra de perguntas frequentes, próximas aulas, histórico, plano, TutorIA, Mentoria, gravações elegíveis, financeiro (para o responsável, por aluno vinculado) e notificações |
| Painel do professor | Gerir a atividade | Solicitações recebidas, agenda, preço, materiais, acompanhamentos e avaliações |
| Denúncias e regras de convivência | Proteger a comunidade | Botão "Denunciar" no perfil do professor, nas avaliações e nas mensagens da Mentoria, janela com motivo e detalhes, lista "Minhas denúncias" e página Regras de convivência |

| Tela | Função | Elementos essenciais |
|---|---|---|
| Área administrativa | Operar a plataforma | Candidaturas, qualificações, perguntas frequentes (criar, editar, desativar e apagar), moderação de denúncias e indicadores |

Fonte: Elaborado pelos autores (2026).

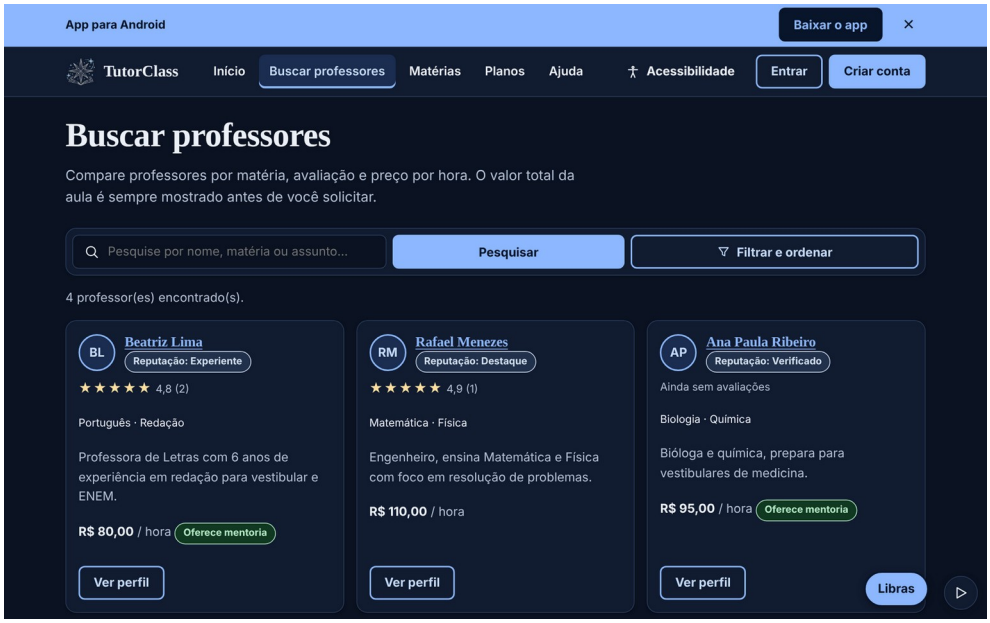
A interface implementada é ilustrada nas Figuras 1 a 9, capturadas em tema escuro, em ambiente de demonstração, com a API simulada em memória e dados fictícios. A página inicial reúne a busca por matéria, o acesso ao painel de acessibilidade e o vídeo decorativo de fundo (Figura 1); a busca apresenta a barra de pesquisa, o botão de filtros e cartões com preço e reputação (Figura 2); o cadastro de aluno traz o aceite dos Termos de uso e da Política de privacidade (Figura 3); o painel do aluno mostra a barra lateral, com a Visão geral e os demais módulos, e a barra de perguntas frequentes (Figura 4); a TutorIA responde em *streaming* (Figura 5); a Mentoria abre a conversa com o professor mentor e permite denunciar uma mensagem (Figura 6); o painel de preferências permite ajustar o tamanho do texto, o tema, o contraste e o movimento (Figura 7); a janela de denúncia pede o motivo e os detalhes (Figura 8); e a fila de moderação reúne as denúncias recebidas pela administração (Figura 9). A versão para celular, que reorganiza o cabeçalho, o menu e as colunas, é apresentada na seção 5.2.

**Figura 1 – Página inicial da interface Web, em tema escuro**



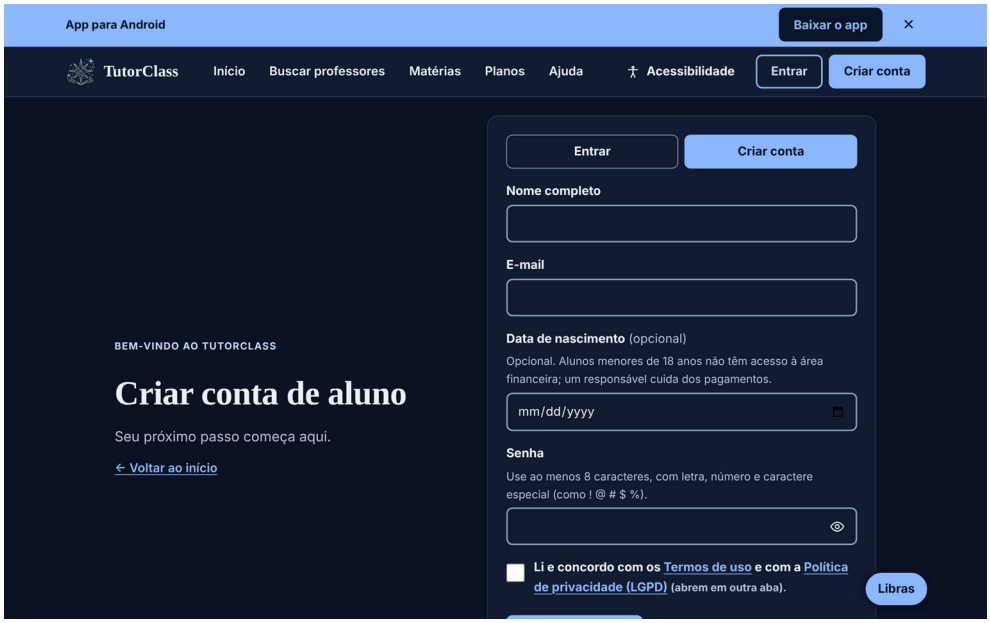

Fonte: Elaborado pelos autores (2026), em ambiente de demonstração com dados fictícios.

**Figura 2 – Busca de professores com filtros e cartões de resultado**




Fonte: Elaborado pelos autores (2026), em ambiente de demonstração com dados fictícios.

**Figura 3 – Cadastro de aluno com o aceite dos Termos de uso e da Política de privacidade**



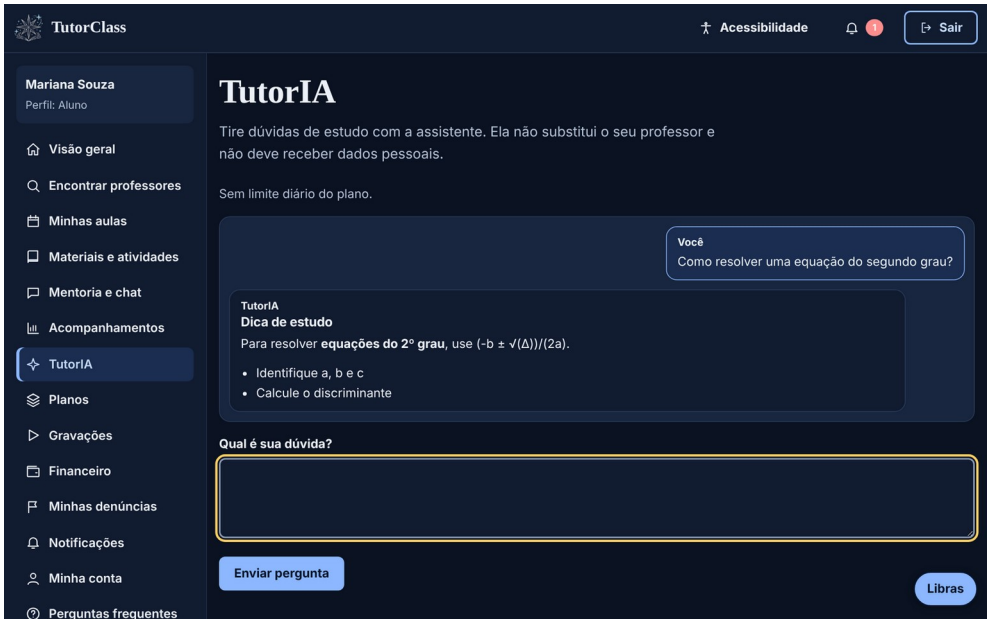

Fonte: Elaborado pelos autores (2026), em ambiente de demonstração com dados fictícios.

**Figura 4 – Painel do aluno, com barra lateral e perguntas frequentes**



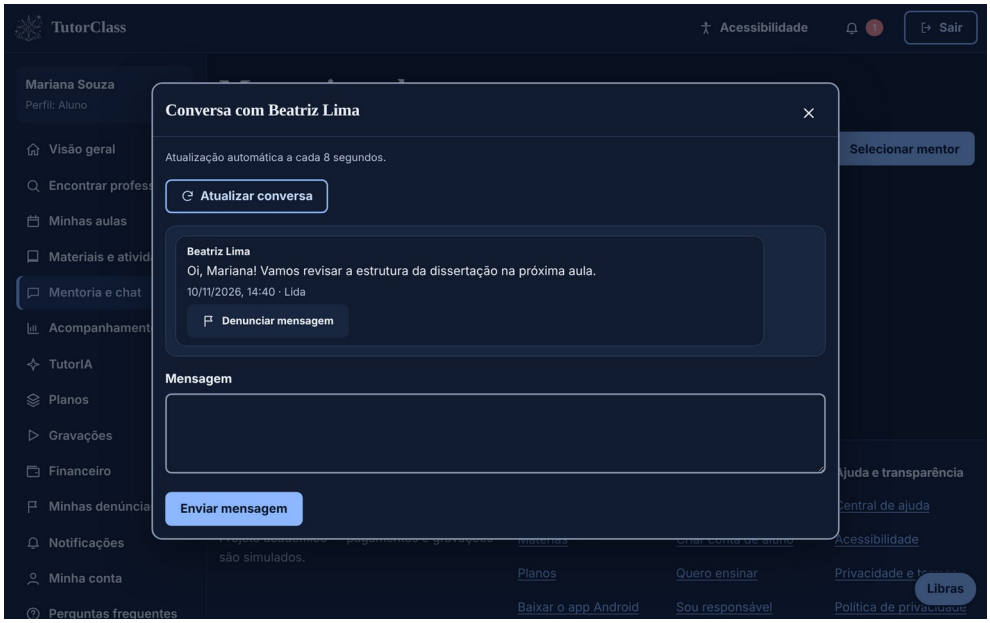

Fonte: Elaborado pelos autores (2026), em ambiente de demonstração com dados fictícios.

**Figura 5 – TutorIA no painel do aluno, com resposta em streaming**




Fonte: Elaborado pelos autores (2026), em ambiente de demonstração com dados fictícios.

**Figura 6 – Conversa de Mentoria com a opção de denunciar uma mensagem**



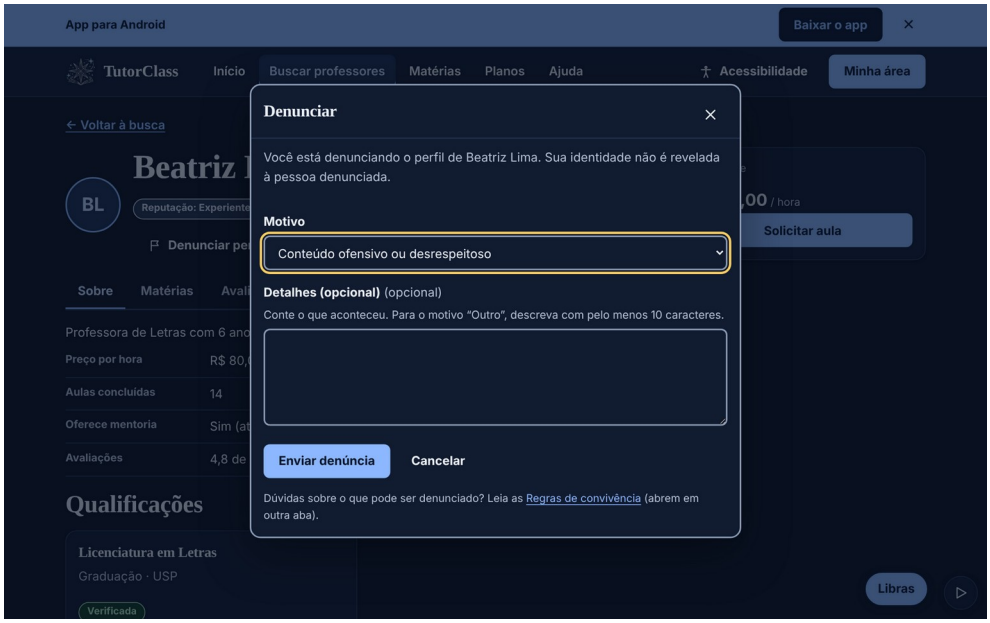

Fonte: Elaborado pelos autores (2026), em ambiente de demonstração com dados fictícios.

**Figura 7 – Painel de preferências de acessibilidade, com a escolha de tema**



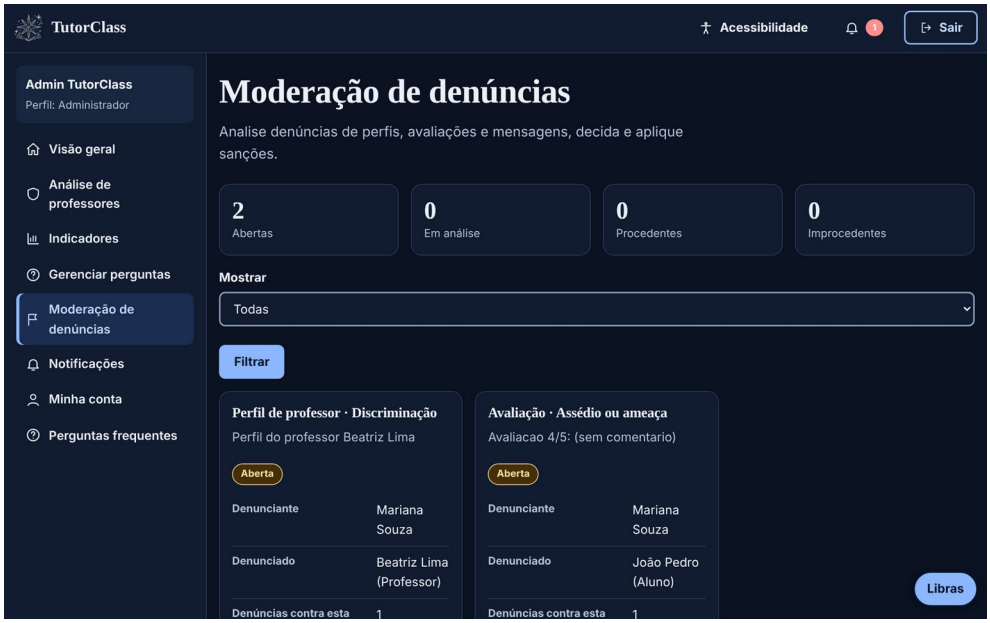

Fonte: Elaborado pelos autores (2026), em ambiente de demonstração com dados fictícios.

**Figura 8 – Janela de denúncia de um perfil de professor**




Fonte: Elaborado pelos autores (2026), em ambiente de demonstração com dados fictícios.

**Figura 9 – Moderação de denúncias na área administrativa**




Fonte: Elaborado pelos autores (2026), em ambiente de demonstração com dados fictícios.

Observa-se que a proposta inicial de interface previa a negociação de preço entre aluno e professor. No back-end implementado, porém, o preço da aula é calculado pela própria plataforma a partir do preço por hora do professor e da duração solicitada e fica registrado na aula, preservando o histórico caso o professor altere o preço no futuro (TUTORCLASS, 2026). A interface foi, portanto, alinhada a esse comportamento, e a negociação direta permanece como possível evolução.

### 4.3 APIs REST, autenticação, segurança e integração entre módulos

A comunicação entre a interface Web e o back-end ocorre pela API REST do TutorClass, organizada por recursos sob o prefixo /api. A Tabela 5 agrupa os principais pontos de acesso implementados, com o perfil exigido, e o serviço expõe ainda o endpoint de verificação de saúde /health e a documentação interativa Swagger/OpenAPI.

**Tabela 5 – Principais grupos de endpoints da API REST**

| Grupo | Rotas principais | Acesso |
|---|---|---|
| Autenticação | POST /api/auth/registrar/aluno \| responsavel \| professor; POST /api/auth/login; GET /api/auth/me | Público; me autenticado |
| Professores e matérias | GET /api/professores; GET /api/professores/{id}; GET /api/professores/{id}/horarios-livres; GET /api/materias | Público |
| Perfil docente | POST /api/professores/{id}/materias \| qualificacoes \| disponibilidades; POST /api/professores/{id}/candidaturas | Professor |
| Aulas | POST /api/aulas; GET /api/aulas/minhas; POST /api/aulas/{id}/confirmar \| concluir; POST /api/aulas/{id}/cancelar; PUT /api/aulas/{id}/reagendar | Aluno, professor ou responsável, conforme a ação |
| Avaliação e acompanhamento | POST /api/aulas/{id}/avaliacao; PUT /api/aulas/{id}/acompanhamento; GET /api/professores/{id}/avaliacoes | Aluno; professor; consulta pública |
| Planos e financeiro | GET /api/planos/me; POST /api/planos/me/assinatura; POST /api/financeiro/aulas/{id}/pagar; GET /api/financeiro/me/historico | Aluno ou responsável |
| TutorIA, FAQ e notificações | POST /api/tutoria/chat; GET /api/faq; POST /api/faq/chat; GET /api/notificacoes | Aluno (TutorIA); FAQ público; autenticado |
| Mentoria | POST /api/mentorias/me; GET /api/mentorias/{vinculoId}/chat/mensagens; hub SignalR /hubs/mentoria-chat | Aluno, professor ou responsável autorizado |
| Responsável | GET /api/responsavel/alunos; POST /api/responsavel/alunos/{id}/aulas; GET /api/responsavel/alunos/{id}/plano; GET /api/responsavel/alunos/{id}/financeiro/historico; POST /api/responsavel/alunos/{id}/aulas/{aulaId}/pagar | Responsável |
| Denúncias e moderação | POST /api/denuncias; GET /api/denuncias/minhas; GET /api/admin/denuncias; GET /api/admin/denuncias/resumo; POST /api/admin/denuncias/{id}/iniciar-analise \| decidir \| reativar-conta | Autenticado (denunciar); Administrador (moderar) |
| Administração | POST /api/admin/candidaturas-professor/{id}/aprovar \| reprovar \| iniciar-analise; POST /api/admin/qualificacoes/{id}/verificar; /api/admin/faq (inclui DELETE); /api/admin/analytics/* | Administrador |

Fonte: Elaborado pelos autores (2026), a partir dos controllers da API.

A autenticação utiliza *tokens* JWT: o login emite o token, que o cliente envia no cabeçalho Authorization: Bearer das requisições protegidas. A API valida emissor, audiência,

assinatura e validade do token, cuja expiração configurada é de 120 minutos, e aplica políticas de autorização por perfil (Aluno, Responsável, Professor e Administrador). A política de senhas exige ao menos 8 caracteres, com letra, número e caractere especial, validada no navegador e novamente na API, e a data de nascimento informada no cadastro não pode ser futura nem anterior a 120 anos, também conferida nas duas camadas. Senhas são tratadas com o *hasher* do ASP.NET Core Identity, e o segredo do JWT, as conexões de banco e as chaves externas ficam em variáveis de ambiente, fora do código e dos clientes. A API também aplica redirecionamento para HTTPS fora do desenvolvimento, CORS restrito às origens configuradas e um *middleware* global de tratamento de exceções. Como o controle por perfil não dispensa verificações de vínculo e de propriedade, aulas, gravações e conversas de Mentoria são conferidas no servidor, independentemente de botões ou menus ocultos na interface. As práticas seguem as recomendações do projeto OWASP (OWASP, [s.d.]).

O cliente Web trata os estados de carregamento, lista vazia e erro nas telas que consomem a API, valida os dados no navegador (experiência) e no servidor (segurança) e pagina as listas extensas. O token JWT é mantido apenas em memória, a resposta da TutorIA chega por *streaming* (*Server-Sent Events*) e é exibida como texto, nunca como HTML, e o chat de Mentoria consulta novas mensagens periodicamente, a cada 8 segundos, pelos endpoints REST, o que simplifica a hospedagem; a comunicação em tempo real por SignalR permanece disponível no back-end como evolução. Na área logada, o cabeçalho reúne a marca, à esquerda, e os botões de acessibilidade, notificações e saída, à direita, enquanto a barra lateral concentra a Visão geral, as aulas, os materiais e os demais módulos de cada perfil; a busca de professores combina campo de pesquisa e gaveta de filtros no topo; o administrador pode editar, desativar ou apagar perguntas do FAQ, e as perguntas frequentes estão disponíveis, com campo de pesquisa na tela inicial, para todos os perfis; o responsável acompanha o financeiro de cada aluno vinculado; o aluno avalia o professor após uma aula concluída, pelo cartão da aula ou pelo perfil do professor; quem está no plano Gratuito e tenta usar um recurso de plano superior (TutorIA, gravações ou Mentoria) recebe um aviso amigável no topo da tela, que convida a conhecer o plano seguinte; alunos menores de 18 anos, identificados pela data de nascimento do cadastro, têm o acesso à área financeira bloqueado com aviso na tela; e contas suspensas pela moderação deixam de ser aceitas em qualquer requisição, mesmo com o token ainda válido. Os avisos aparecem no topo, com botão para fechar e desaparecem em 4 segundos, com pausa ao passar o mouse ou focar. Nas páginas públicas, uma faixa fixa no topo, uma seção na página inicial, o rodapé e a Central de ajuda (com o passo a passo de instalação) convidam a baixar o aplicativo Android; esse convite não aparece dentro do próprio aplicativo nem em aparelhos

iOS, para os quais o arquivo não serve, e a faixa pode ser fechada. Em relação a pagamentos, o projeto não utiliza gateway real; em uma operação comercial, seria necessário adotar um gateway certificado em vez de manipular dados de cartão.

A integração entre módulos ocorre no back-end. O plano contratado determina o acesso à TutorIA, à Mentoria e às gravações; eventos de aula, como solicitação, confirmação, cancelamento e reagendamento, geram notificações internas para os envolvidos; e a Mentoria associa o vínculo relacional, mantido no PostgreSQL, ao histórico de mensagens, mantido no MongoDB, com a verificação de permissão feita pela API antes de qualquer leitura ou envio.

### 4.4 Estrutura administrativa

A área administrativa reúne as operações restritas ao perfil Administrador. A primeira é a análise de candidaturas de professores, que percorre os estados Pendente, Em análise, Aguardando documentação, Aprovada e Reprovada, com as ações de iniciar análise, aprovar, reprovar e solicitar complementação. A segunda é a verificação das qualificações apresentadas pelo professor, que podem ser verificadas ou rejeitadas. A terceira é a moderação de denúncias de perfis, avaliações e mensagens (seção 3.2), com fila por situação, início da análise, decisão com justificativa, advertência, suspensão e reativação de contas. Há também o cadastro, a edição, a ativação, a desativação e a exclusão das perguntas do FAQ/ChatBot e o registro manual das referências de gravação de aulas, que, nesta versão acadêmica, são simuladas.

Por fim, o módulo de análise de dados oferece indicadores agregados, como total de alunos e professores, aulas concluídas e canceladas, avaliação média, assinaturas por plano, matéria mais procurada e receitas simuladas. Inclui ainda uma previsão de demanda por matéria, feita por um classificador supervisionado simples, de caráter demonstrativo, que usa apenas dados estruturados do PostgreSQL e não acessa mensagens privadas.

### 4.5 UX e UI Design

A experiência do usuário (UX) abrange toda a interação da pessoa com o produto, incluindo utilidade, facilidade, confiança e satisfação, enquanto a interface (UI) é a camada visual e interativa. Garrett (2002) organiza o trabalho de UX em camadas, da estratégia e do escopo até a estrutura, o esqueleto e a superfície. A Tabela 6 apresenta as personas, definidas como hipóteses iniciais a serem validadas com entrevistas e questionários (COOPER et al., 2014).

**Tabela 6 – Personas hipotéticas do TutorClass**

| Persona | Contexto e objetivos | Dores e necessidades |
|---|---|---|
| Mariana, 17 anos, aluna do 3º ano | Estuda em escola pública, prepara-se para o vestibular e o ENEM e usa o celular como principal dispositivo; quer reforçar Matemática e Redação com orçamento limitado. | Não sabe se o preço é justo e desconfia de perfis sem avaliações. Precisa de filtro por preço, avaliações visíveis e horários flexíveis. |
| Carlos, 45 anos, pai de aluno | Quer contratar reforço para o filho de 15 anos e acompanhar as aulas, acessando por celular e computador. | Teme golpes e tem dificuldade para comparar professores. Precisa de perfis verificados, preço claro e histórico de aulas. |
| Beatriz, 24 anos, professora recém-formada em Letras | Quer ampliar a renda com aulas online de Português e Redação, definindo seu preço e sua agenda. | Tem dificuldade de divulgação e de organizar solicitações. Precisa de perfil atrativo, painel de solicitações, regras de cancelamento e avaliações. |

Fonte: Elaborado pelos autores (2026).

O fluxo principal do aluno parte da Home, com a busca por matéria; segue para os resultados, onde aplica filtros e ordenação; abre o perfil do professor, com avaliações, qualificações e agenda; solicita a aula escolhendo data, horário e duração, com o preço calculado e exibido antes da confirmação; aguarda a confirmação do professor; e, após a aula concluída, avalia o profissional. O fluxo do professor inicia no cadastro e na candidatura, passa pela definição de matérias, preço por hora e disponibilidade e segue para o recebimento de solicitações, a confirmação, a conclusão da aula e o registro do acompanhamento. Entre as exceções previstas estão o usuário não autenticado que tenta solicitar uma aula (encaminhado ao login e devolvido à mesma tela), a busca sem resultados (com sugestão de ampliar os filtros) e o cancelamento ou reagendamento de aulas.

A organização das telas partiu de wireframes de baixa fidelidade, descritos para o celular e depois implementados diretamente em HTML e CSS. A Home reúne logotipo, campo de busca em destaque, atalhos de matérias e a chamada para o aplicativo; os resultados aparecem em cartões com iniciais do professor, nome, matérias, nota, preço por hora e o botão “Ver perfil”; o perfil do professor usa abas (Sobre, Matérias, Avaliações e Agenda) e o botão “Solicitar aula”; e o agendamento mostra os horários livres, o resumo do valor e a confirmação. No computador, a área logada tem barra lateral com os módulos de cada perfil e a busca ganha uma coluna adicional de cartões; no celular, a barra lateral dá lugar a uma barra de navegação inferior com cinco atalhos (por exemplo, Início, Professores, Aulas, TutorIA e Menu) e os cartões passam para uma coluna. Um protótipo de média ou alta fidelidade em ferramenta de design, como o Figma, não foi produzido: a interface implementada (Figuras 1 a 12) cumpre o papel de protótipo funcional e é a base da avaliação de usabilidade.

A camada de UI foi construída sobre um *design system* mínimo, em CSS próprio e sem bibliotecas externas, com variáveis de cor, tipografia (fonte de destaque para títulos e de leitura para o texto), espaçamento e arredondamento. Três recursos de aparência ficam à escolha da pessoa no painel de acessibilidade: o tema (seguir o sistema, claro ou escuro), o contraste (padrão ou alto) e a escala do texto (100%, 115% ou 130%), além da opção de reduzir animações; as escolhas são guardadas no navegador e valem para todas as telas, e as capturas deste documento foram feitas em tema escuro. Os componentes são reutilizados em toda a interface: cartões, *chips* de matéria, selos de situação, botões primário, secundário e discreto, janelas de diálogo, avisos temporários, listas paginadas e avaliação por estrelas. Cada tela tem um botão principal, o foco do teclado é visível, todos os campos têm rótulo e o preço é mostrado com o valor por hora e o total da aula antes da confirmação. Os cadastros e as denúncias usam textos curtos e links para os documentos que explicam o que será feito com os dados. A análise de usabilidade parte das heurísticas de Nielsen (1994), aplicadas ao TutorClass na Tabela 7.

**Tabela 7 – Heurísticas de Nielsen aplicadas ao TutorClass**

| **Heurística** | **Aplicação na plataforma** |
|---|---|
| Visibilidade do status do sistema | Mostrar o estado da aula (solicitada, confirmada, concluída ou cancelada) e o carregamento das consultas. |
| Correspondência com o mundo real | Usar a linguagem do aluno: “reforço”, “aula avulsa”, “vestibular”. |
| Controle e liberdade do usuário | Permitir cancelar e reagendar aulas e desfazer filtros. |
| Consistência e padrões | Mesmos componentes e termos em todas as telas. |
| Prevenção de erros | Confirmação antes de cancelar, assinar um plano, decidir uma denúncia ou reativar uma conta; aceite explícito dos Termos no cadastro; validação da duração da aula (entre 30 e 240 minutos), da data de nascimento (sem datas futuras) e limite diário de denúncias. |
| Reconhecimento em vez de memorização | Manter filtros ativos visíveis e o histórico de aulas à mão. |
| Flexibilidade e eficiência | Atalhos de matérias e filtros salvos. |
| Design estético e minimalista | Evitar excesso de informação nos cartões. |
| Ajuda no reconhecimento e correção de erros | Mensagens claras que indiquem o problema e como corrigi-lo. |
| Ajuda e documentação | FAQ/ChatBot de ajuda integrado à plataforma. |

Fonte: Elaborado pelos autores (2026).

A verificação de qualidade da interface foi feita em três frentes. A primeira é a auditoria automática com o axe-core (WCAG 2.2 A e AA e boas práticas), que percorreu 168 páginas e estados da interface Web em quatro condições (desktop de 1280 px, largura de 320 px, tema escuro e alto contraste) e não encontrou violações na última execução; os testes de ponta a ponta percorrem os fluxos principais de cada perfil. A segunda é uma simulação de tarefas por

*script*, em navegador automatizado e com a API simulada, que segue o caminho mínimo de cinco tarefas e conta as interações (cliques, escolhas e digitações): encontrar um professor de Matemática com preço de até R$ 110 por hora (6 interações), solicitar uma aula a ele (5), denunciar uma mensagem inadequada na Mentoria (5), definir preço e apresentação como professor (4) e decidir uma denúncia como administrador (6). As cinco tarefas foram concluídas, cada uma em no máximo seis interações, o que mostra que os fluxos existem e são curtos; mas a simulação representa uma pessoa que já conhece o caminho e não mede tempo humano, erros nem satisfação, de modo que não substitui o teste com usuários. A terceira frente foi um teste de usabilidade exploratório com cinco pessoas, todas do círculo próximo da equipe: duas integrantes do grupo e uma conhecida que não conhecia o site, as três de 20 a 22 anos; a mãe de um integrante, de 36 anos; e o irmão de uma integrante, um adolescente de 13 anos. O teste foi feito em computador com Windows e em celular, pelo navegador e pelo aplicativo Android, e os fluxos de todos os perfis da plataforma foram percorridos. O processo transcorreu sem intercorrências e ninguém relatou dificuldade para concluir as tarefas. Registraram-se três observações: uma participante de 20 a 22 anos elogiou a acessibilidade do sistema, principalmente na versão Web em celular; a participante de 36 anos gostou de contar com o aplicativo; e o adolescente de 13 anos notou que o cadastro aceitava datas de nascimento no futuro, falha já corrigida (a data futura agora é recusada na tela e também na API). Além dela, o único defeito encontrado foi o widget VLibras abrir sozinho, também corrigido. O teste tem limites que devem constar: a amostra é pequena, por conveniência e formada por pessoas com vínculo pessoal com a equipe (familiares e conhecidos), o que pode atenuar críticas; as duas integrantes do grupo conhecem o sistema, o que pode favorecer o desempenho, e só uma participante não o conhecia previamente; não houve medição formal de tempo nem de taxa de sucesso por tarefa, nem aplicação da escala SUS (BROOKE, 1996); e não participaram pessoas com deficiência nem usuárias de tecnologias assistivas. Os resultados indicam, portanto, a ausência de barreiras evidentes nos fluxos principais, mas não permitem generalização. Como evolução, recomenda-se repetir o teste com cinco a oito participantes por perfil, incluindo mais pessoas de outras faixas etárias e que usam leitores de tela, com a técnica de pensar em voz alta e medição de taxa de sucesso, tempo, número de erros e satisfação pela escala SUS, em que pontuações acima de 68 são consideradas acima da média (SAURO, 2011); cinco usuários revelam a maior parte dos problemas em testes qualitativos (NIELSEN, 2000). Como hipóteses a verificar nesse teste ampliado, apontam-se o excesso de filtros em tela pequena, a dependência de avaliações e verificação de perfil para gerar confiança, o risco de abandono em cadastros

longos e a facilidade de encontrar o botão de denúncia. Os resultados da auditoria automática não substituem a avaliação com pessoas e tecnologias assistivas.

# 5 DESENVOLVIMENTO DA SOLUÇÃO MOBILE

## 5.1 Objetivo, tecnologias e arquitetura

O aplicativo Mobile do TutorClass oferece, em dispositivos Android, o mesmo conjunto de funcionalidades da versão Web: autenticação, cadastro, os quatro perfis de usuário, acessibilidade e tradução para Libras. Os dados são acessados exclusivamente pela API, que valida permissões e aplica as regras de negócio.

O aplicativo foi desenvolvido em C# com .NET MAUI, com alvo Android 6 ou superior. Na etapa final do projeto, optou-se por uma arquitetura híbrida: uma casca nativa abre, em um componente *WebView*, a própria interface Web do TutorClass, servida pela API na mesma origem. A decisão evita manter duas interfaces com o mesmo conteúdo, garante que o aplicativo e o site tenham sempre o mesmo comportamento e visual, e herda os recursos de acessibilidade já auditados. As camadas nativas anteriores, em padrão Model–View–ViewModel (MVVM), permanecem no projeto com seus testes. As responsabilidades de cada componente estão na Tabela 8. A aplicação utiliza os endpoints do back-end ASP.NET Core publicado no Render, que acessa o PostgreSQL/Neon e, para o chat da Mentoria, o MongoDB Atlas. O fluxo é: aplicativo, requisição HTTPS, API, serviços de negócio e bancos de dados, com a resposta retornando pelo mesmo caminho.

**Tabela 8 – Camadas e componentes do aplicativo Mobile**

| Camada / componente | Responsabilidade |
|---|---|
| Casca nativa (WebShellPage) | Abre a interface Web em uma WebView, com tela de carregamento, aviso de falta de conexão e nova tentativa, botão Voltar do Android e abertura de links externos no navegador. |
| Interface Web | Telas, navegação, acessibilidade, VLibras e consumo da API, idênticos ao site; carregada de https://tutorclass.onrender.com/. |
| Camadas nativas anteriores (Views, ViewModels, Services, Models) | Telas nativas de Login, Home e demais áreas, serviços HTTP, armazenamento seguro e DTOs, mantidos com seus testes, mas fora do fluxo inicial. |
| Back-end ASP.NET Core | Autorização, regras de negócio, persistência e exposição dos endpoints. |

Fonte: Elaborado pelos autores (2026).

## 5.2 Telas e navegação

A versão Mobile contempla as áreas listadas na Tabela 9, as mesmas da interface Web, incluindo as denúncias e a página de regras. A navegação segue a da Web, com histórico: o botão Voltar do Android retorna à tela anterior e, na primeira tela, fecha o aplicativo. No celular,

a navegação principal passa para uma barra inferior, com os módulos mais usados de cada perfil e o botão Menu, que abre os demais. Como o aplicativo carrega a própria interface Web, as telas seguem o tema escolhido pela pessoa, e as Figuras 10 a 12 apresentam o tema escuro: a entrada, o painel do aluno e a conversa de Mentoria (Figura 10); a busca, a janela de denúncia e o acompanhamento das denúncias enviadas (Figura 11); e os painéis do professor e da administração, com a página inicial (Figura 12). As imagens foram geradas com a interface em modo de aplicativo (identificação TutorClassApp) e dimensões de celular, com a API simulada e dados fictícios.

**Tabela 9 – Telas e áreas do aplicativo Mobile**

| Tela / área | Finalidade |
|---|---|
| Abertura | Exibir a logo e o andamento do carregamento, avisando quando o servidor está iniciando. |
| Entrar e Criar conta | Autenticar o usuário ou cadastrar aluno, responsável ou professor pela API. |
| Página inicial e busca | Buscar professores por matéria, consultar perfis, preços, avaliações e planos. |
| Painéis por perfil | Aluno, responsável, professor e administrador, com Visão geral, aulas, materiais, financeiro e demais módulos. |
| TutorIA e Mentoria | Acessar a IA educacional e o chat de mentoria, conforme o plano contratado. |
| Perguntas frequentes e Ajuda | Consultar e pesquisar o FAQ, disponível para todos os perfis. |
| Denúncias e Regras de convivência | Denunciar perfis, avaliações e mensagens, acompanhar as denúncias enviadas e consultar as regras; a administração modera a fila. |
| Acessibilidade e VLibras | Ajustar texto, tema, contraste e movimento e traduzir o conteúdo para Libras. |
| Sem conexão | Informar a falta de internet e permitir tentar novamente. |

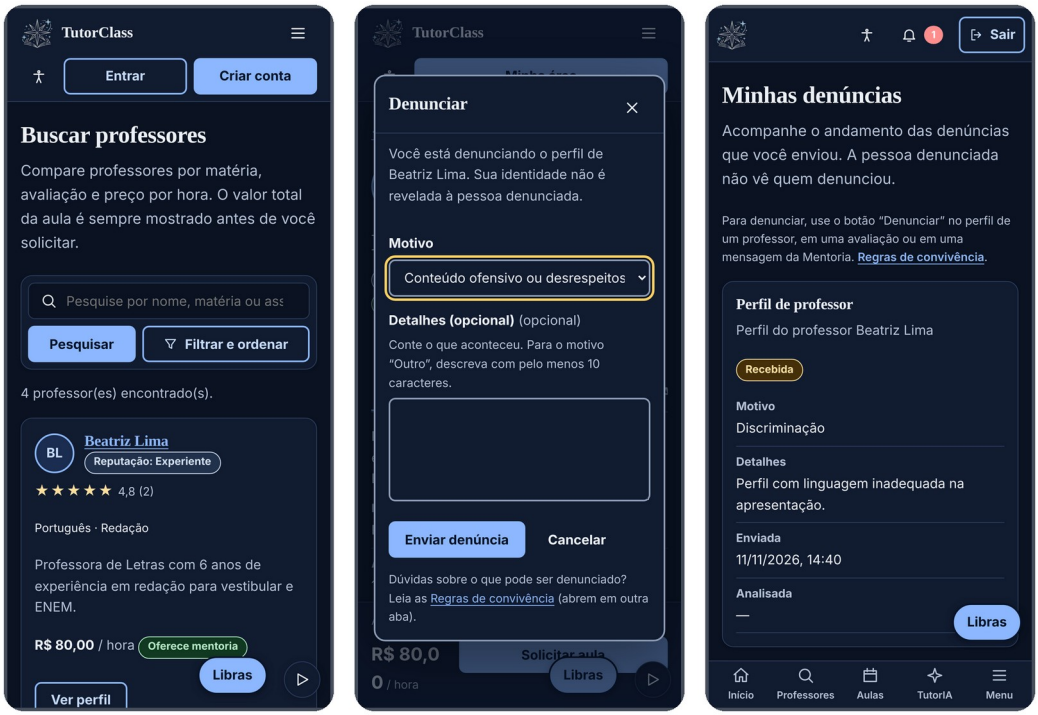
Fonte: Elaborado pelos autores (2026).

**Figura 10 – Aplicativo Mobile em tema escuro: entrada, painel do aluno e Mentoria**



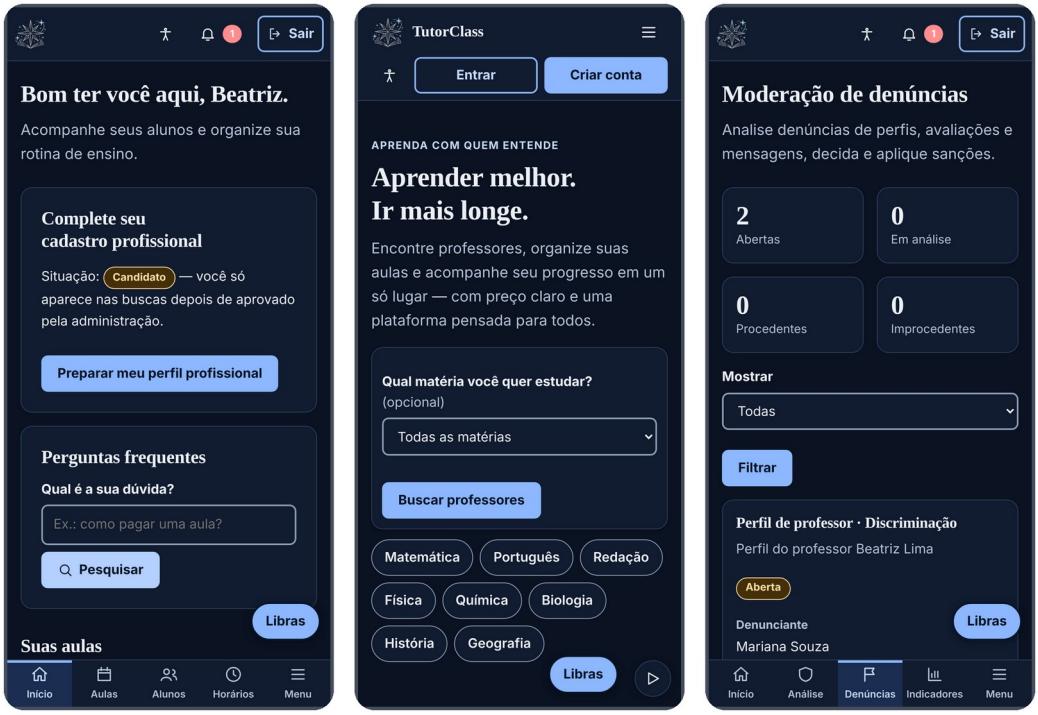

Fonte: Elaborado pelos autores (2026), capturas em tema escuro, com a API simulada e dados fictícios.

**Figura 11 – Aplicativo Mobile em tema escuro: busca, janela de denúncia e minhas denúncias**




Fonte: Elaborado pelos autores (2026), capturas em tema escuro, com a API simulada e dados fictícios.

**Figura 12 – Aplicativo Mobile em tema escuro: painel do professor, página inicial e moderação**




Fonte: Elaborado pelos autores (2026), capturas em tema escuro, com a API simulada e dados fictícios.

## 5.3 Autenticação, integração e sincronização de dados

O fluxo de autenticação utiliza POST /api/auth/login e, em seguida, GET /api/auth/me para obter os dados do usuário autenticado. Na arquitetura híbrida, a autenticação ocorre na interface Web carregada pelo aplicativo: o token JWT fica apenas na memória da página, a senha não é persistida e, ao fechar o aplicativo, o usuário precisa entrar novamente. A comunicação é somente por HTTPS, e conteúdo misto é bloqueado. As camadas nativas anteriores usam o *SecureStorage* do Android para guardar o token. Em ambos os casos, a autorização continua sendo feita no servidor, e o aplicativo não contém senhas de banco, *strings* de conexão ou chaves privadas.

A integração consome a API REST por requisições HTTPS com dados em JSON, e o back-end devolve as respostas necessárias para a atualização das telas. A sincronização é, portanto, sob demanda: a cada abertura ou atualização de tela, o aplicativo consulta a API. Não há sincronização offline, banco local ou acesso direto aos bancos, e essas características não são apresentadas como implementadas; sem internet, o aplicativo exibe o aviso de falta de conexão.

## 5.4 Testes e estado da implementação

O histórico do projeto registra testes automatizados de serviços e ViewModels e a validação real de login e de chamada autenticada à API hospedada. Na etapa final, o aplicativo

foi compilado em modo de depuração (.NET 10, JDK 21), instalado em um celular Android físico e conferido pela equipe quanto à tela de entrada, ao cadastro, à acessibilidade, ao VLibras, ao botão Voltar e ao estado sem conexão. Dentro do aplicativo, a interface abre por padrão em tema escuro (a pessoa pode trocá-lo no painel de acessibilidade) e a faixa de divulgação do aplicativo não é exibida. A cor do aplicativo (barra superior, tela de abertura, ícone e telas de carregamento) segue o azul da interface Web. A Tabela 10 resume o estado de cada aspecto.

**Tabela 10 – Estado da implementação e das evidências do aplicativo Mobile**

| Aspecto | Estado / evidência |
|---|---|
| Compilação Android | Build de depuração concluído e pacote APK gerado, com .NET 10 e JDK 21. |
| Execução em celular físico | Instalado e conferido pela equipe: entrada, cadastro, acessibilidade, VLibras, botão Voltar e estado sem conexão. |
| Autenticação e chamada à API | Testes de integração com o back-end remoto registrados; autenticação também exercida pela interface Web. |
| Camadas nativas anteriores | Testes de serviços e ViewModels mantidos; fora do fluxo inicial. |
| Distribuição do APK | Arquivo TutorClass.apk anexado a uma Release do GitHub e oferecido por botões no site (faixa, página inicial, rodapé e Central de ajuda); a instalação exige permitir fontes desconhecidas no Android. |
| Denúncias e moderação no aplicativo | Disponíveis pela interface Web carregada no aplicativo, verificadas em modo de aplicativo (Figuras 10 a 12) e conferidas em celular físico pela equipe, sem pendências. |
| Teste de usabilidade com usuários | Realizado em caráter exploratório com cinco pessoas (de 13 a 36 anos), em Windows e em celular (navegador e aplicativo), sem intercorrências; dois defeitos encontrados, o VLibras que abria sozinho e a data de nascimento futura no cadastro, foram corrigidos (seção 4.5). Ampliação com medição formal e SUS pendente. |
| Compatibilidade com iOS | Sem suporte: o aplicativo é apenas para Android. No iPhone, o acesso é pela interface Web, no navegador e sem instalação; o convite para baixar o aplicativo não é exibido em aparelhos iOS. |
| Publicação em loja | Não realizada; o pacote gerado é de desenvolvimento e distribuído por Release do GitHub. |

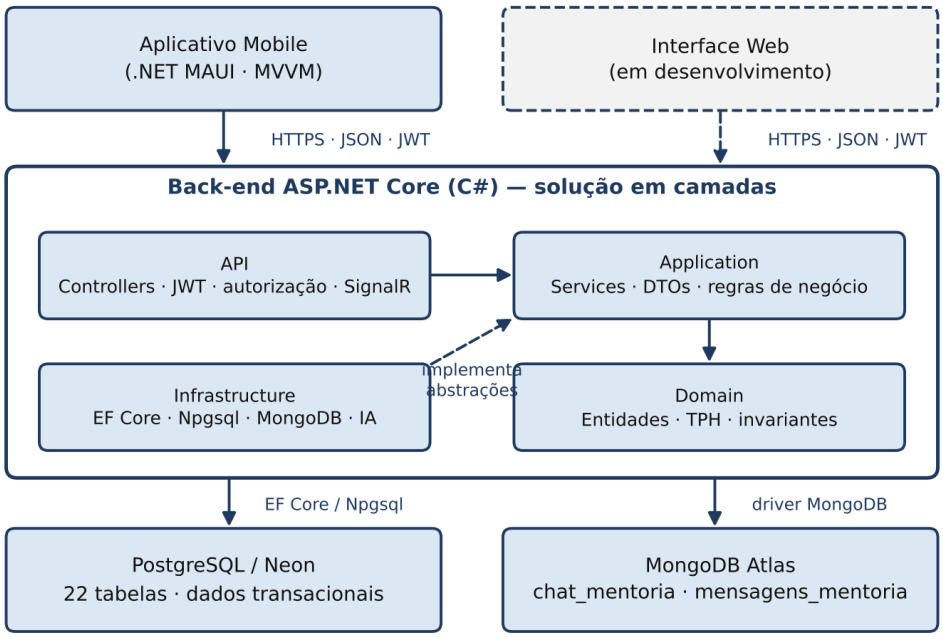
Fonte: Elaborado pelos autores (2026).

# 6 ARQUITETURA DE SOFTWARE

## 6.1 Visão geral e organização em camadas

A solução é organizada em interfaces clientes, uma API central e serviços de persistência. O back-end é desenvolvido em C# com ASP.NET Core Web API; a persistência relacional utiliza PostgreSQL hospedado no Neon e o histórico de mensagens do chat de Mentoria utiliza MongoDB Atlas. As interfaces consomem a API por HTTPS e não se conectam diretamente aos bancos, e a API concentra autenticação, autorização e execução das regras de negócio. Essa separação permite que os clientes Web e Mobile compartilhem serviços sem duplicar regras de domínio nas interfaces. A Figura 13 representa a estrutura geral.

**Figura 13 – Arquitetura em camadas e comunicação entre os componentes do TutorClass**




Fonte: Elaborado pelos autores (2026).

O back-end está dividido em camadas com responsabilidades distintas, descritas na Tabela 11. A divisão reduz o acoplamento entre as regras do TutorClass e os detalhes de interface ou armazenamento.

**Tabela 11 – Camadas da solução e suas responsabilidades**

| Camada | Responsabilidade na arquitetura |
|---|---|
| Domain | Entidades, especializações de usuário e conceitos centrais, como aula, candidatura de professor, assinatura e Mentoria. |
| Application | Serviços e casos de uso que coordenam operações, validações e regras de negócio. |

| Camada | Responsabilidade na arquitetura |
|---|---|
| Infrastructure | Acesso a dados e integrações com PostgreSQL/Neon, MongoDB Atlas e serviços externos usados pela aplicação. |
| API | Exposição de endpoints HTTP, autenticação, autorização e configuração da aplicação. |
| Tests | Projetos de testes para validação de comportamentos do back-end e do aplicativo, conforme a cobertura implementada. |

Fonte: Elaborado pelos autores (2026).

## 6.2 Orientação a objetos e modularização

O back-end aplica orientação a objetos na representação dos perfis e das funcionalidades educacionais. O Usuário é a base conceitual dos perfis Aluno, Responsável, Professor e Administrador; no banco relacional, esses perfis são persistidos na tabela de usuários pela estratégia TPH (*Table Per Hierarchy*), identificada por um discriminador de tipo, o que não implica uma tabela física para cada perfil. O domínio também utiliza encapsulamento e invariantes: o estado de uma aula tem escrita privada e só muda por comportamentos como confirmar, cancelar e concluir; a duração deve estar entre 30 e 240 minutos e o preço acordado deve ser maior que zero; e a candidatura de um professor só muda de estado por métodos próprios, o que impede alterações arbitrárias.

As responsabilidades são agrupadas em módulos funcionais: autenticação e perfis; professores e validação de qualificações; matérias e aulas; planos e pagamentos simulados; Mentoria e comunicação; TutorIA; e materiais, gravações e notificações.

## 6.3 Fluxo de uma operação, comunicação e segurança

Em uma solicitação de aula, a interface cliente envia uma requisição HTTP à API; a API identifica o usuário e verifica as permissões; a camada de aplicação valida os dados e coordena a operação; e a infraestrutura persiste ou consulta os registros no PostgreSQL, devolvendo a resposta pela API. Entre as regras validadas estão a de que apenas professores aprovados recebem aulas, a de que o professor leciona a matéria e a de que o horário está dentro da disponibilidade e sem sobreposição.

A comunicação é protegida por autenticação JWT e autorização por perfil, e as operações administrativas e os recursos restritos por plano são validados no back-end. No chat da Mentoria, o SignalR provê a comunicação em tempo real, enquanto o histórico é mantido no MongoDB. A arquitetura apresentada não implica auditoria de segurança completa nem garantia de ausência de vulnerabilidades.

# 7 PROJETO DO BANCO DE DADOS

A persistência do TutorClass utiliza dois bancos com responsabilidades distintas. O Neon, baseado em PostgreSQL, é o banco relacional principal e armazena os dados estruturados e transacionais. O MongoDB Atlas é utilizado de forma complementar para as conversas do módulo de Mentoria. A separação mantém no PostgreSQL as regras de negócio e os relacionamentos que exigem integridade referencial, enquanto as mensagens do chat são persistidas como documentos.

## 7.1 Modelo lógico e MER relacional (PostgreSQL/Neon)

O modelo lógico foi elaborado a partir da estrutura efetivamente implementada no back-end e contém 23 tabelas físicas, com chave primária id em todas elas. Os perfis Aluno, Responsável, Professor e Administrador são especializações da entidade Usuário no domínio, mas são persistidos na tabela usuarios por herança TPH, com o campo tipo_usuario como discriminador. As Tabelas 12 a 15 descrevem as tabelas por área funcional, com suas finalidades e chaves estrangeiras principais, e a Figura 14 apresenta o modelo entidade-relacionamento.

**Tabela 12 – Usuários, professores e matérias**

| Tabela | Finalidade | Chaves estrangeiras principais |
|---|---|---|
| usuarios | Usuários e dados específicos dos perfis persistidos por TPH. | — |
| responsaveis_alunos | Vínculo entre responsáveis e alunos. | responsavel_id e aluno_id → usuarios |
| materias | Matérias oferecidas na plataforma. | — |
| professores_materias | Matérias que cada professor leciona. | professor_id → usuarios; materia_id → materias |
| candidaturas_professor | Processo de candidatura e análise do professor. | professor_id → usuarios |
| qualificacoes_professor | Formações e qualificações apresentadas pelo professor. | professor_id → usuarios |
| disponibilidades_professor | Dias e horários disponibilizados pelo professor. | professor_id → usuarios |

Fonte: Elaborado pelos autores (2026).

**Tabela 13 – Aulas e recursos pedagógicos**

| Tabela | Finalidade | Chaves estrangeiras principais |
|---|---|---|
| aulas | Aulas contratadas entre aluno e professor. | aluno_id, professor_id e solicitada_por_usuario_id → usuarios; materia_id → materias |

| Tabela | Finalidade | Chaves estrangeiras principais |
|---|---|---|
| aulas_reagendamentos | Histórico de alterações de horário das aulas. | aula_id → aulas; solicitado_por_usuario_id → usuarios |
| acompanhamentos_aula | Acompanhamento pedagógico relacionado a uma aula. | aula_id → aulas; aluno_id e professor_id → usuarios |
| avaliacoes_professor | Avaliação feita pelo aluno após a aula. | aula_id → aulas; aluno_id e professor_id → usuarios |
| materiais | Metadados de materiais disponibilizados ao aluno. | aluno_id e professor_id → usuarios; materia_id → materias; aula_id → aulas (opcional) |
| gravacoes_aula | Metadados e URL da gravação associada a uma aula. | aula_id → aulas |

Fonte: Elaborado pelos autores (2026).

**Tabela 14 – Planos, financeiro, TutorIA e Mentoria**

| Tabela | Finalidade | Chaves estrangeiras principais |
|---|---|---|
| assinaturas_aluno | Plano contratado para o aluno e usuário pagador. | aluno_id e pagador_usuario_id → usuarios |
| cobrancas_assinatura | Cobranças mensais de uma assinatura. | assinatura_id → assinaturas_aluno; pagador_usuario_id → usuarios |
| pagamentos_aula | Pagamento da aula, comissão da plataforma e valor líquido do professor. | aula_id → aulas; aluno_id, professor_id e pagador_usuario_id → usuarios |
| usos_ia_aluno | Contagem diária de mensagens usadas na TutorIA. | aluno_id → usuarios |
| mentorias_aluno_professor | Vínculo de Mentoria entre aluno e professor. | aluno_id e professor_id → usuarios |
| acompanhamentos_mentoria | Acompanhamento pós-aula feito pelo mentor. | vinculo_mentoria_id → mentorias_aluno_professor; aula_id → aulas; aluno_id e professor_id → usuarios |
| adicionais_mentoria | Adicional financeiro simulado por acompanhamento concluído. | acompanhamento_mentoria_id → acompanhamentos_mentoria; aula_id → aulas; aluno_id e professor_id → usuarios |

Fonte: Elaborado pelos autores (2026).

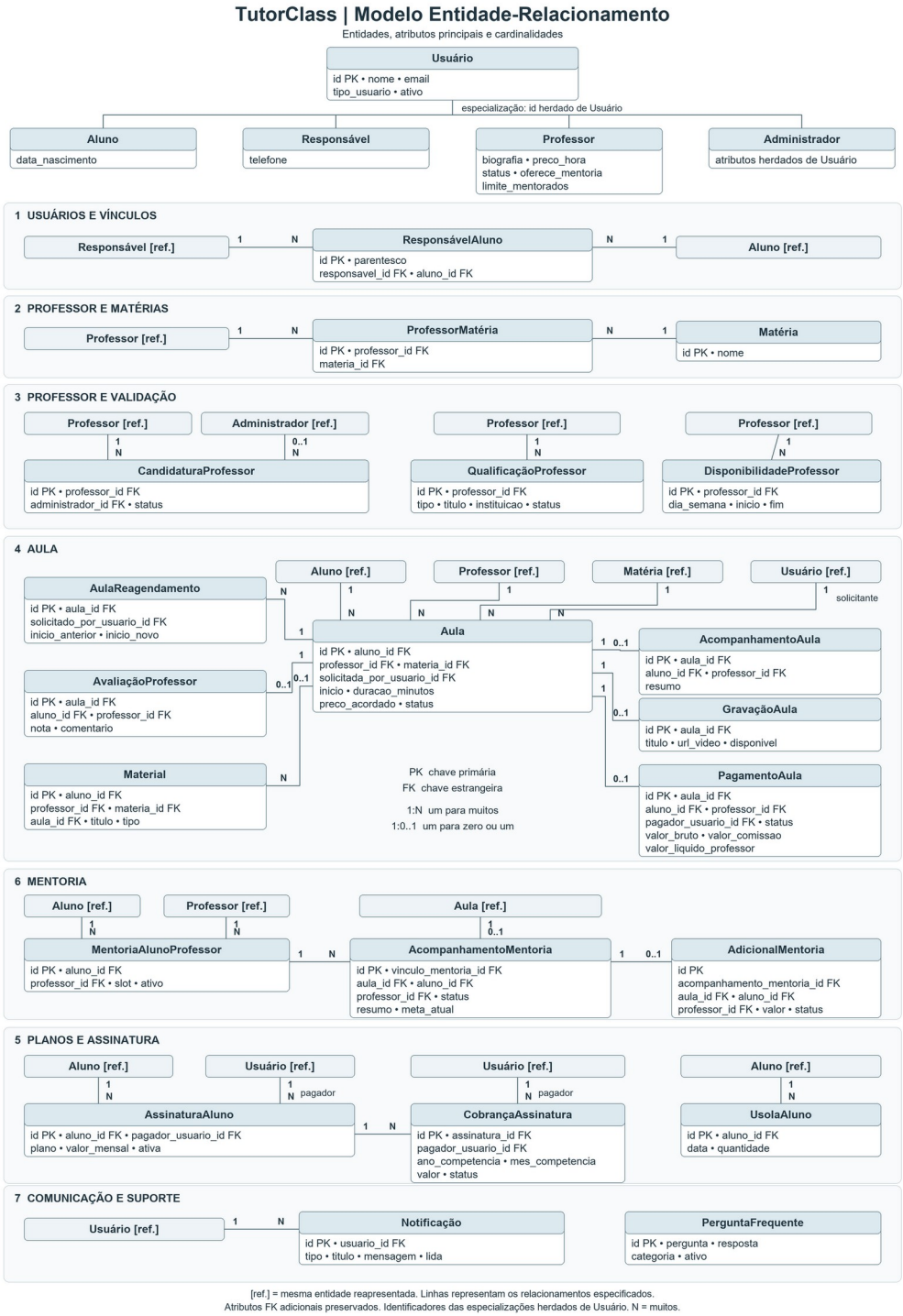
**Tabela 15 – Comunicação e suporte**

| Tabela | Finalidade | Chaves estrangeiras principais |
|---|---|---|
| notificacoes | Notificações internas destinadas aos usuários. | usuario_id → usuarios |
| perguntas_frequentes | Perguntas e respostas usadas pelo FAQ/ChatBot. | — |
| denuncias | Denúncias de perfis, avaliações e mensagens, com motivo, situação, sanção e decisão da moderação. | denunciante_id e alvo_usuario_id → usuarios |

Fonte: Elaborado pelos autores (2026).

As restrições de unicidade reforçam as regras de negócio: e-mail único em usuarios; combinação única entre responsável e aluno e entre professor e matéria; nome único em matérias; no máximo uma avaliação, um acompanhamento, uma gravação e um pagamento por aula; uma cobrança por assinatura e competência mensal; e um registro diário de uso da TutorIA por aluno. Reagendamentos e cobranças têm cardinalidade um-para-muitos. Para o desempenho das consultas mais frequentes, o script cria índices compostos, como os de aulas por aluno ou professor, data de início e status, os de notificações por usuário e situação de leitura os de pagamentos por status e os de denúncias por usuário denunciado e por situação. A tabela denuncias, criada pela migração mais recente, é representada na última figura do Apêndice A e guarda o alvo da denúncia (tipo e identificador, com um resumo do conteúdo denunciado), o motivo, a situação (aberta, em análise, procedente ou improcedente), a sanção aplicada, a justificativa e o administrador responsável.

**Figura 14 – Modelo entidade-relacionamento do TutorClass (PostgreSQL/Neon)**




Fonte: Elaborado pelos autores (2026).

## 7.2 Modelo documental e integração com o MongoDB Atlas

O MongoDB Atlas é usado exclusivamente no módulo de chat da Mentoria. Diferentemente do PostgreSQL, sua estrutura não usa chaves estrangeiras, e a associação com

os registros relacionais é feita pela aplicação, por meio dos identificadores do vínculo de Mentoria e dos usuários. A Tabela 16 descreve as duas *collections*.

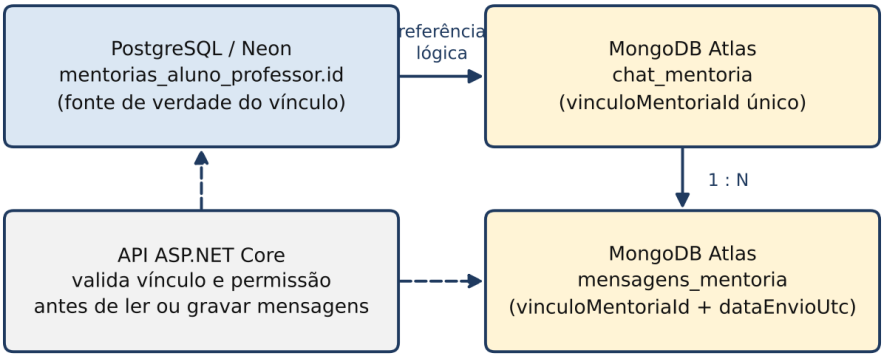
**Tabela 16 – Collections do MongoDB Atlas**

| Collection | Campos principais | Índices | Função |
|---|---|---|---|
| chat_mentoria | _id; vinculoMentoriaId; alunoId; professorId; dataCriacaoUtc | vinculoMentoriaId (único) | Chat associado a um vínculo de Mentoria. |
| mensagens_mentoria | _id; chatMentoriaId; vinculoMentoriaId; remetenteUsuarioId; remetenteNome; conteudo; dataEnvioUtc; lida; dataLeituraUtc | vinculoMentoriaId + dataEnvioUtc; chatMentoriaId + dataEnvioUtc | Mensagens trocadas entre aluno e professor mentor. |

Fonte: Elaborado pelos autores (2026).

A tabela mentorias_aluno_professor, no PostgreSQL, é a fonte de verdade do vínculo entre aluno e mentor. Seu identificador é gravado no campo vinculoMentoriaId da collection chat_mentoria, como mostra a Figura 15, e o back-end consulta o banco relacional para verificar se a Mentoria é válida e se o usuário está autorizado antes de permitir a leitura ou o envio de mensagens. A referência entre os bancos é lógica e controlada pela camada de aplicação. A adoção de dois bancos não duplica responsabilidades: o PostgreSQL concentra os dados cadastrais, financeiros e pedagógicos que exigem integridade referencial, e o MongoDB guarda apenas o histórico de comunicação.

**Figura 15 – Integração lógica entre o PostgreSQL/Neon e o MongoDB Atlas**




Fonte: Elaborado pelos autores (2026).

## 7.3 Procedures, triggers e script consolidado

Além das tabelas, o banco no Neon recebeu rotinas complementares para o fluxo de análise de candidaturas de professores, descritas na Tabela 17. Elas completam a estrutura relacional com operações e validações executadas no próprio banco: as procedures conduzem a

candidatura pelas etapas de análise e os *triggers* recusam alterações inconsistentes, mesmo quando feitas diretamente no banco e não pela API. O back-end em C# aplica as mesmas regras de transição e de motivo obrigatório; as rotinas funcionam, portanto, como salvaguarda de integridade e como operação de apoio à administração, e não substituem a API. O script inclui ainda *triggers* de integridade para as denúncias, que exigem administrador e justificativa nas denúncias decididas e permitem sanção apenas quando a denúncia é procedente.

**Tabela 17 – Procedures, funções e triggers das candidaturas de professores e das denúncias**

| Objeto | Tipo | Função |
|---|---|---|
| iniciar_analise_candidatura | Procedure | Move a candidatura de Pendente para Em análise, registrando o administrador e a data da análise. |
| finalizar_analise_candidatura | Procedure | Finaliza a análise como Aprovada, Reprovada ou Aguardando documentação, exigindo motivo nos dois últimos casos. |
| solicitar_documentacao_candidatura | Procedure | Registra o pedido de documentação ou justificativa ao professor, com motivo obrigatório. |
| trg_candidaturas_atualizar_updated_at | Trigger | Atualiza automaticamente a data de modificação a cada alteração da candidatura. |
| trg_validar_transicao_candidatura | Trigger | Impede que a candidatura passe de Pendente direto a um estado final, sem antes estar Em análise. |
| trg_validar_motivo_candidatura | Trigger | Exige motivo preenchido quando o status é Reprovada ou Aguardando documentação. |
| trg_denuncias_atualizar_updated_at | Trigger | Atualiza a data de modificação a cada alteração de uma denúncia. |
| trg_validar_decisao_denuncia | Trigger | Exige administrador e justificativa em denúncias procedentes ou improcedentes e permite sanção apenas em denúncia procedente. |

Fonte: Elaborado pelos autores (2026).

O script consolidado do banco (TutorClass_Script_Consolidado_PIM_IV.sql) reúne três partes: o histórico de migrations gerado pelo Entity Framework Core, com as tabelas, os índices, a tabela de denúncias e uma visão de consulta administrativa (vw_professores); as procedures e os *triggers* das candidaturas; e os *triggers* de integridade das denúncias, todos listados na Tabela 17. O arquivo está no repositório Git do projeto (pasta database/scripts) e é idempotente: controla as migrations aplicadas pela tabela __EFMigrationsHistory e usa CREATE OR REPLACE e DROP TRIGGER IF EXISTS, de modo que pode ser reexecutado sem duplicar objetos. Foi executado em uma instalação limpa do PostgreSQL 16 e reexecutado sem erros, e as regras das procedures e dos *triggers* foram exercitadas com dados de teste (transições inválidas, motivo ausente e sanção indevida são recusados). Em produção, as tabelas são criadas pelas migrations do back-end; o script não contém as *collections* e os índices do

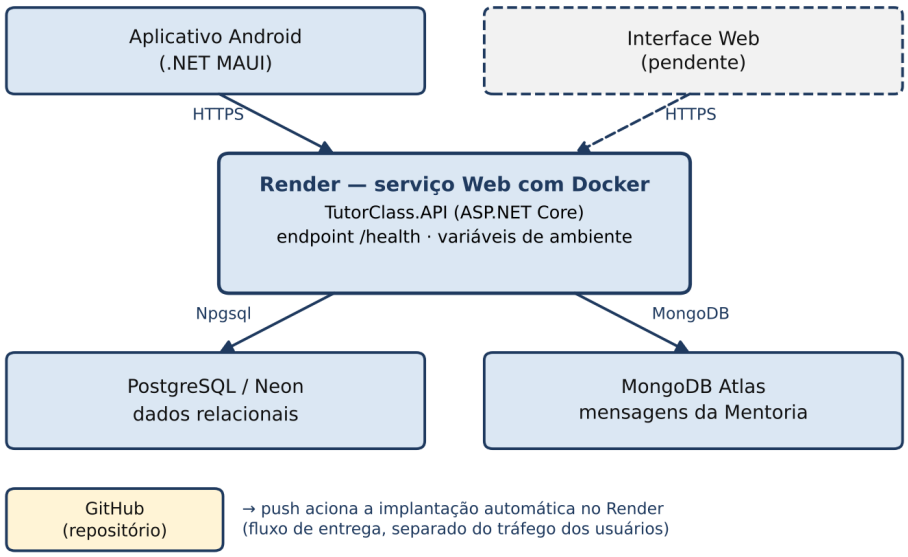
MongoDB, e sua aplicação em uma instância existente deve ser precedida de cópia de segurança. A representação do modelo físico, com tipos, chaves e restrições das tabelas, está no Apêndice A.

# 8 INFRAESTRUTURA EM NUVEM E DEVOPS

## 8.1 Arquitetura em nuvem e serviços utilizados

O TutorClass utiliza uma arquitetura cliente-servidor em que o aplicativo Mobile e a interface Web consomem, por HTTPS, os serviços do back-end ASP.NET Core Web API. O back-end centraliza regras de negócio, autenticação e autorização e se comunica com o PostgreSQL hospedado no Neon e com o MongoDB Atlas; os clientes não acessam os bancos diretamente. A Figura 16 mostra a implantação, e a Tabela 18 descreve os serviços e a situação de cada um.

**Figura 16 – Arquitetura de implantação e serviços em nuvem do TutorClass**




Fonte: Elaborado pelos autores (2026).

**Tabela 18 – Serviços de infraestrutura e situação**

| Componente | Função no TutorClass | Situação |
|---|---|---|
| Render | Hospedagem e execução da API ASP.NET Core em serviço Web com Docker. | Implementado |
| Neon / PostgreSQL | Persistência relacional de cadastros, aulas, pagamentos simulados e assinaturas. | Implementado |
| MongoDB Atlas | Persistência dos documentos de chat e mensagens da Mentoria. | Implementado |
| GitHub | Versionamento do código-fonte e repositório que fornece a versão implantada. | Implementado |
| GitHub Desktop | Operação local de commit e push para o repositório. | Fluxo de trabalho utilizado |

| Componente | Função no TutorClass | Situação |
|---|---|---|
| Interface Web | Cliente em HTML, CSS e JavaScript, servido pela própria API e consumindo os endpoints HTTPS. | Implementado, executado contra a API real e validado em teste exploratório com cinco pessoas |
| VLibras | Widget do Governo Federal que traduz o conteúdo para Libras, carregado do serviço oficial quando a pessoa aciona o botão “Libras”. | Implementado; depende de serviço externo |
| Aplicativo Android | Casca nativa em .NET MAUI que abre a interface Web, integrada à API REST. | Implementado e conferido em celular físico; APK distribuído por Release do GitHub |

Fonte: Elaborado pelos autores (2026).

### 8.2 Contêineres, versionamento e fluxo de entrega

A API é disponibilizada como serviço Web no Render, com implantação baseada em Docker. O Dockerfile do projeto usa uma imagem de compilação com o SDK do .NET e uma imagem final apenas com o ambiente de execução do ASP.NET, o que reúne a aplicação e suas dependências e permite executar o back-end sem instalar manualmente as ferramentas de desenvolvimento no servidor. O arquivo de configuração do serviço define a verificação de saúde em /health e as variáveis de produção.

O código é versionado com Git e hospedado no GitHub. O fluxo consiste em desenvolver e validar as alterações localmente, registrar um *commit*, enviá-lo ao repositório remoto e deixar que o Render faça a implantação automática associada às mudanças. Esse mecanismo automatiza a disponibilização da API, mas não equivale a um *pipeline* completo de integração contínua com testes automáticos a cada alteração, o que é uma melhoria futura. Antes de cada publicação, a equipe executa localmente a suíte de testes do back-end e do aplicativo (542 testes, todos aprovados na última execução, incluindo os das denúncias e da moderação), os testes do front-end (módulos JavaScript), um teste de ponta a ponta em navegador e a auditoria automática de acessibilidade, scripts mantidos no repositório.

### 8.3 Segurança, monitoramento e escalabilidade

A configuração de produção usa variáveis de ambiente para a conexão ao PostgreSQL, a conexão ao MongoDB e o segredo do JWT; chaves de serviços externos também ficam fora do código-fonte e de documentos públicos, e este relatório não reproduz valores reais de senhas, tokens ou *strings* de conexão.

Para a disponibilidade e o monitoramento, o back-end expõe o endpoint /health, que informa o estado do serviço e a situação das conexões com o PostgreSQL e o MongoDB, e

registra *logs* de aplicação que auxiliam a identificar falhas. O plano gratuito de hospedagem pode apresentar demora na resposta após inatividade, o que exige conferir a disponibilidade antes de demonstrações.

A separação entre clientes, API e serviços de persistência permite evoluir cada componente individualmente. Como possibilidades de evolução, e não como recursos implementados, destacam-se a ampliação da hospedagem, a otimização de consultas, *pipelines* automáticos de testes e monitoramento com alertas, considerando limites do plano e custos.

# 9 GERENCIAMENTO ÁGIL DO PROJETO

## 9.1 Organização do trabalho e Product Backlog

O TutorClass é desenvolvido de forma incremental, com separação de responsabilidades entre back-end, aplicativo Mobile e interface Web. Esta seção adota os artefatos do Scrum (SCHWABER; SUTHERLAND, 2020) e o quadro Kanban; os períodos e os responsáveis de cada sprint devem ser preenchidos pelo grupo, pois não há registro formal de cerimônias nesta documentação.

O *Product Backlog* reúne as funcionalidades do produto, incluindo itens desenvolvidos e pendências (Tabela 19).

**Tabela 19 – Product Backlog**

| ID | Item do produto / entrega | Situação documentável |
|---|---|---|
| PB-01 | Cadastro, login e autenticação JWT | Back-end implementado |
| PB-02 | Consulta de professores e matérias | Back-end implementado |
| PB-03 | Agendamento e acompanhamento de aulas | Back-end e interface Web implementados |
| PB-04 | Assinaturas e controle de benefícios por plano | Back-end e interface Web implementados |
| PB-05 | Chat de Mentoria e histórico | Back-end e interface Web implementados (chat por consulta periódica) |
| PB-06 | TutorIA, FAQ e notificações | Back-end e interface Web implementados |
| PB-07 | Aplicativo Mobile e integração com a API | Implementado e conferido em celular físico |
| PB-08 | Interface Web, navegação e integração com a API | Interface implementada, verificada com API simulada e executada contra a API real |
| PB-09 | Modelagem de dados e documentação do PIM IV | Documentação atualizada com o projeto pronto; revisão final do grupo concluída |
| PB-10 | Testes integrados Web–API e evidências finais | Suíte .NET (542 testes), testes do front-end, teste de ponta a ponta e auditoria automática de acessibilidade executados; teste de usabilidade exploratório com cinco pessoas e avaliação manual de acessibilidade (teclado e atalhos do site) realizados, com os defeitos encontrados corrigidos |
| PB-11 | Recursos de plano, perguntas frequentes, financeiro do responsável, avaliação do professor, VLibras, download do aplicativo Android, política de senha, validação de data de nascimento e páginas de privacidade e termos com aceite no cadastro | Interface Web implementada |

| ID | Item do produto / entrega | Situação documentável |
|---|---|---|
| PB-12 | Denúncias, moderação e Regras de convivência (web, aplicativo, API e banco) | Back-end, interface Web e aplicativo implementados, com testes automatizados; conferido em celular físico pela equipe |

Fonte: Elaborado pelos autores (2026).

## 9.2 Sprint Backlog, cronograma e quadro Kanban

O *Sprint Backlog* é o recorte do *Product Backlog* selecionado para uma sprint; a Tabela 20 é um exemplo de planejamento da próxima sprint, e não uma atividade já executada.

**Tabela 20 – Sprint Backlog proposto para a próxima etapa**

| Item | Tarefa da sprint proposta | Critério de conclusão |
|---|---|---|
| PB-10 | Executar o teste manual dos fluxos de cada perfil na API publicada e registrar os resultados | Resultados e problemas encontrados registrados |
| PB-10 | Realizar a avaliação manual de acessibilidade (teclado e leitores de tela) | Relatório com falhas encontradas e correções |
| PB-07 | Planejar e aplicar o teste de usabilidade do aplicativo Mobile com usuários | Roteiro, resultados e ajustes registrados |
| PB-09 | Revisar a documentação com as evidências obtidas | Texto compatível com as funcionalidades efetivamente entregues |

Fonte: Elaborado pelos autores (2026).

O cronograma da Tabela 21 indica a sequência e a duração das entregas por sprint. Os períodos são contados em semanas a partir do início do ciclo e seguem prazos usuais de projetos ágeis: sprints curtas, dentro do limite de um mês recomendado pelo Guia do Scrum (SCHWABER; SUTHERLAND, 2020), e uma semana final de reserva para imprevistos. As datas absolutas e os responsáveis serão fixados pela equipe na reunião de planejamento.

**Tabela 21 – Cronograma de sprints (planejamento sugerido)**

| Sprint | Foco | Entrega esperada | Duração e período |
|---|---|---|---|
| Sprint A | Integração Web–API real | Fluxos principais e suíte .NET executados contra a API (concluída) | 3 semanas (semanas 1 e 3) |
| Sprint B | Acessibilidade manual | Teste com teclado e leitores de tela, relatório de falhas e correções | 1 semana (semana 4) |
| Sprint C | Usabilidade do Mobile e da Web | Roteiro e recrutamento, sessões com pessoas de cada perfil e análise dos resultados | 2 semanas (semanas 5 a 6) |
| Sprint D | Testes e documentação final | Teste manual dos fluxos na API publicada, ajustes dos achados, evidências de integração e relatório revisado | Dias úteis (semana 7) |
| Reserva | Imprevistos e homologação | Folga para correções, nova rodada de testes e ensaio da apresentação | 1 semana (semana 8) |

Fonte: Elaborado pelos autores (2026).

O quadro Kanban da Tabela 22 registra o estado final do acompanhamento: as entregas implementadas e verificadas e, como único item sem atendimento, o aplicativo para iOS, que está fora do escopo atual.

**Tabela 22 – Quadro Kanban: retrato de acompanhamento**

| Concluído / implementado | Em andamento / revisão | A fazer / validar |
|---|---|---|
| Back-end de autenticação JWT e endpoints de professores e matérias | — | Aplicativo para iOS (sem suporte atual) |
| Interface Web para todos os perfis, integrada à API real e publicada no Render | — | — |
| Denúncias, moderação e regras de convivência, com tabela e gatilhos aplicados no banco Neon | — | — |
| Testes do back-end, do front-end e auditoria automática de acessibilidade | — | — |
| Aplicativo Mobile conferido em celular físico, com nova versão do APK publicada no GitHub (Release 1.1) | — | — |
| Teste de usabilidade exploratório com cinco pessoas, avaliação manual de acessibilidade e revisão final da documentação | — | — |

Fonte: Elaborado pelos autores (2026).

## 9.3 Definição de pronto e entregas

Um item só é marcado como concluído quando está integrado ao ramo principal, compila sem erro, atende aos critérios funcionais combinados e possui teste ou demonstração. Para interfaces, inclui-se navegação, consumo real da API, tratamento básico de erros e verificação de acessibilidade; funcionalidades apenas planejadas ou protótipos não são registrados como entregues. As entregas compreendem o back-end e a API, o aplicativo Mobile, a interface Web, o banco de dados com seu script consolidado e esta documentação.

# 10 CONCLUSÃO

Este trabalho documentou o desenvolvimento e a implantação do TutorClass, plataforma que reúne em um só ambiente a busca por professores particulares, a contratação de aulas e o acompanhamento dos recursos de apoio ao estudo. Retomando os objetivos propostos, o modelo de negócio definiu público, proposta de valor, planos e fontes de receita simuladas; a seção de responsabilidade social estabeleceu diretrizes de inclusão e acessibilidade; e as seções técnicas descreveram a solução Web, o aplicativo Mobile, a arquitetura em camadas, a modelagem dos bancos de dados, a infraestrutura em nuvem e o planejamento ágil.

O principal resultado concreto é o back-end em ASP.NET Core, organizado em camadas e integrado ao PostgreSQL/Neon e ao MongoDB Atlas, implantado em contêiner no Render, com autenticação JWT, autorização por perfil, regras de negócio para aulas, planos, Mentoria, TutorIA e notificações, além da interface Web responsiva, desenvolvida com atenção à WCAG 2.2 nível AA e servida pela própria API, executada pela equipe contra a API real, com a tradução para Libras (VLibras), o plano Gratuito com convite ao plano seguinte, o financeiro do responsável, a avaliação do professor, a denúncia e a moderação de conteúdo com as Regras de convivência, e do aplicativo Android em .NET MAUI, que abre essa interface e foi conferido em celular físico. A divisão em dois bancos, com o vínculo de Mentoria como fonte de verdade relacional, e a separação entre clientes e API mostraram-se adequadas para que Web e Mobile compartilhem as mesmas regras sem duplicação.

Permanecem como limitações, declaradas ao longo do texto, a validação ampliada da interface Web com usuários (o teste realizado foi exploratório, com cinco pessoas), a ausência de avaliação com leitores de tela e pessoas com deficiência, a ausência de medição formal e de escala SUS no teste de usabilidade, a ausência de aplicativo para iOS, a configuração de um *pipeline* de integração contínua e a validação comercial do modelo, que depende de pesquisa de mercado, análise de custos e integração com serviços financeiros reais. Como trabalhos futuros, recomendam-se ampliar a validação do aplicativo e da interface Web com usuários, registrar as evidências e evoluir a automação de testes e o monitoramento, consolidando a passagem de um projeto acadêmico para uma solução pronta para uso.

# REFERÊNCIAS


ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**: Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 7 nov. 2026.

BROOKE, John. SUS: a “quick and dirty” usability scale. In: JORDAN, Patrick W. et al. (ed.). **Usability evaluation in industry**. London: Taylor & Francis, 1996.

COOPER, Alan; REIMANN, Robert; CRONIN, David; NOESSEL, Christopher. **About face**: the essentials of interaction design. 4. ed. Indianapolis: Wiley, 2014.

GARRETT, Jesse James. **The elements of user experience**: user-centered design for the web. Indianapolis: New Riders, 2002.

GIL, Antonio Carlos. **Como elaborar projetos de pesquisa**. 6. ed. São Paulo: Atlas, 2017.

MARCONI, Marina de Andrade; LAKATOS, Eva Maria. **Fundamentos de metodologia científica**. 8. ed. São Paulo: Atlas, 2017.

MARCOTTE, Ethan. Responsive web design. **A List Apart**, 2010. Disponível em: https://alistapart.com/article/responsive-web-design/. Acesso em: 7 nov. 2026.

MICROSOFT LEARN. **Documentação do ASP.NET Core**: Web API, autenticação e SignalR. [S. l.]: Microsoft, [s.d.]. Disponível em: https://learn.microsoft.com/aspnet/core/. Acesso em: 7 nov. 2026.

NIELSEN, Jakob. 10 usability heuristics for user interface design. **Nielsen Norman Group**, 1994. Disponível em: https://www.nngroup.com/articles/ten-usability-heuristics/. Acesso em: 7 nov. 2026.

NIELSEN, Jakob. Why you only need to test with 5 users. **Nielsen Norman Group**, 2000. Disponível em: https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/. Acesso em: 7 nov. 2026.

OWASP. **OWASP Top 10**. [S. l.]: OWASP Foundation, [s.d.]. Disponível em: https://owasp.org/www-project-top-ten/. Acesso em: 7 nov. 2026.

SAURO, Jeff. **A practical guide to the System Usability Scale**. [S. l.]: CreateSpace, 2011.

SCHWABER, Ken; SUTHERLAND, Jeff. **The Scrum guide**. [S. l.]: Scrum.org, 2020. Disponível em: https://scrumguides.org/. Acesso em: 7 nov. 2026.

TUTORCLASS. **README e documentação do repositório do projeto**. [S. l.]: Grupo do PIM IV, 2026.

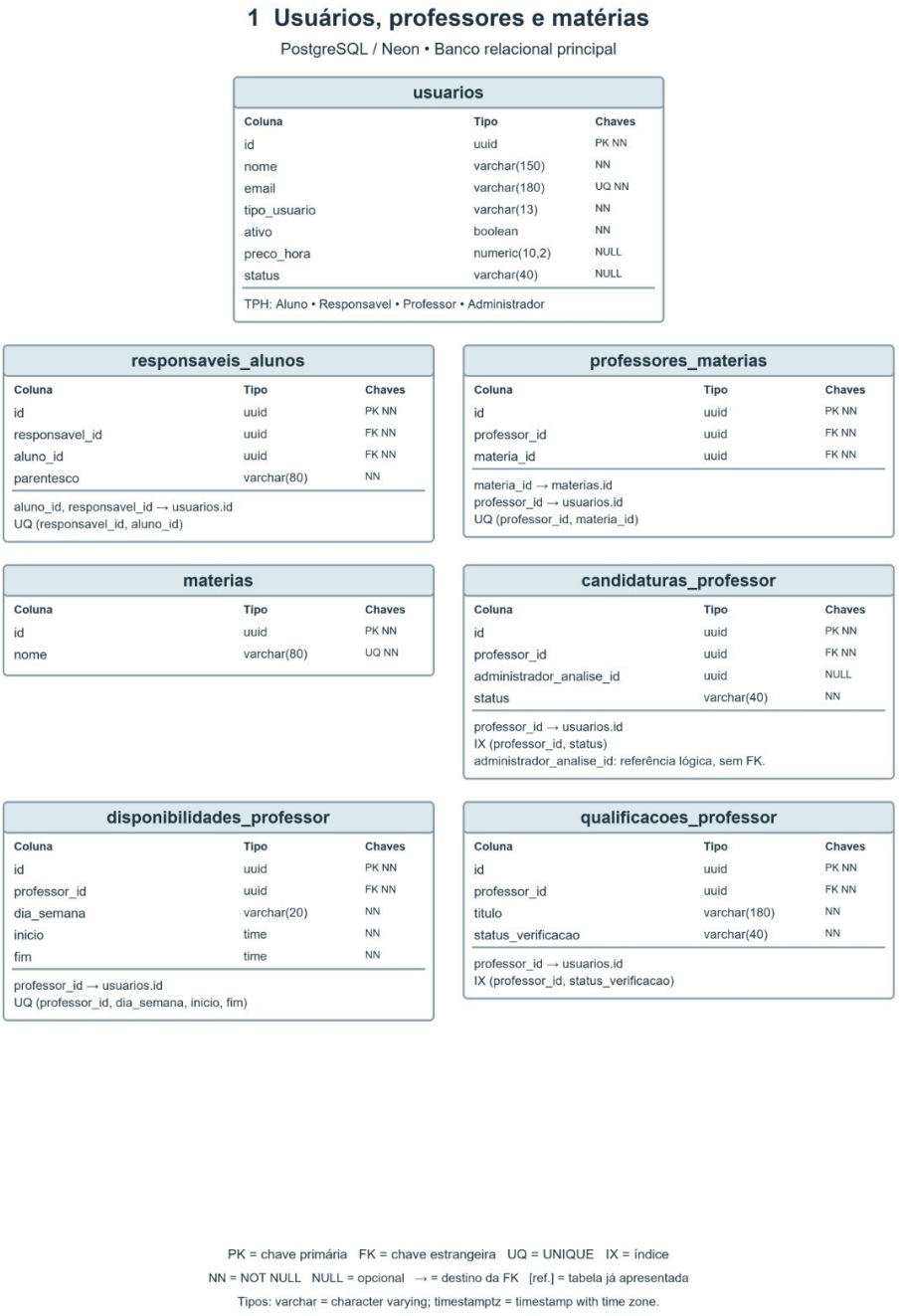
UNIVERSIDADE PAULISTA. **Manual do PIM IV**: Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas. São Paulo: UNIP, [s.d.].

W3C. **Web Content Accessibility Guidelines (WCAG) 2.2**. [S. l.]: W3C, 2023. Disponível em: https://www.w3.org/TR/WCAG22/. Acesso em: 7 nov. 2026.

WROBLEWSKI, Luke. **Mobile first**. New York: A Book Apart, 2011.

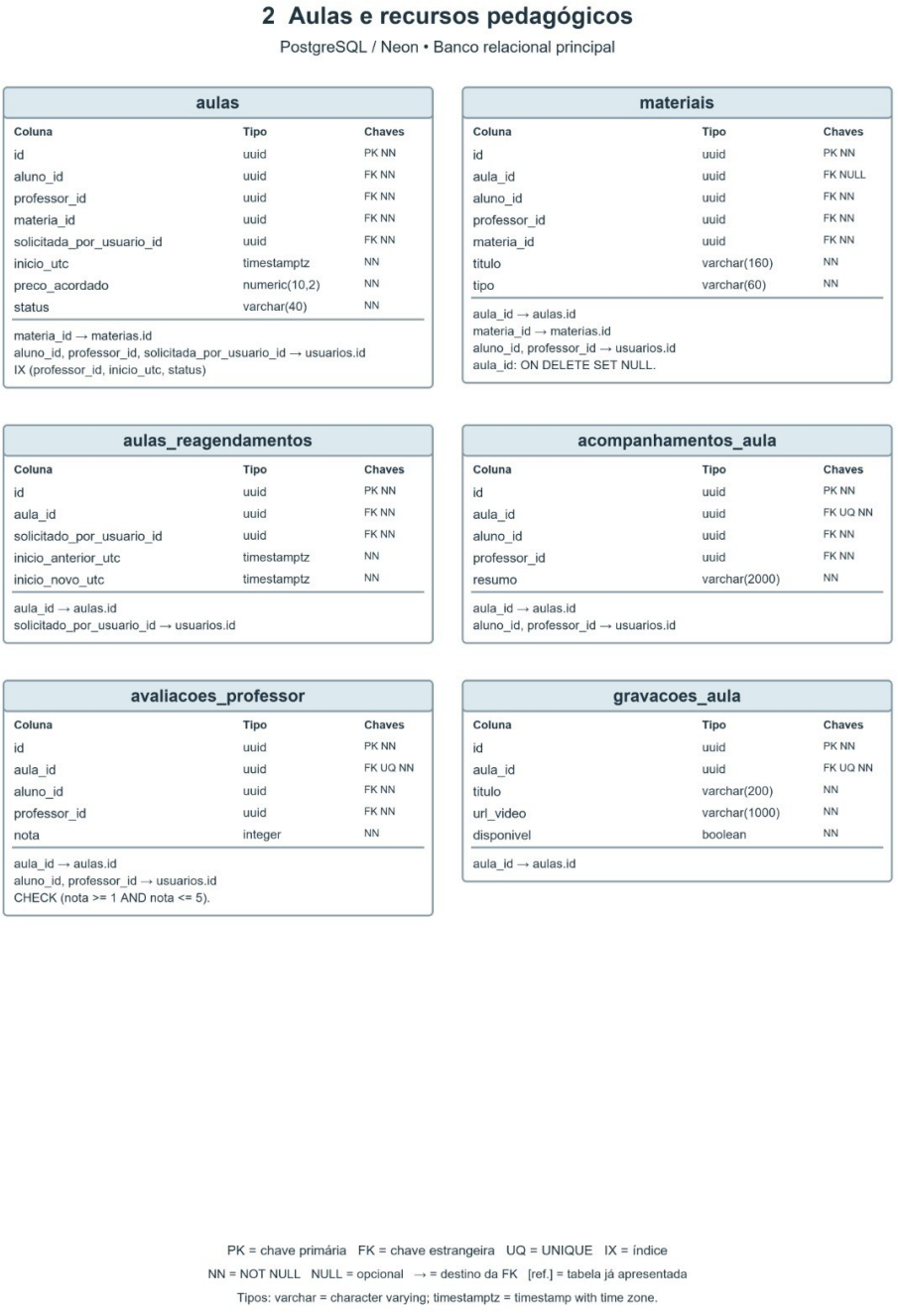
# APÊNDICE A – MODELO FÍSICO DOS BANCOS DE DADOS (POSTGRESQL/NEON E MONGODB ATLAS)

**Figura 17 – Modelo físico (1/6): usuários, professores e matérias**



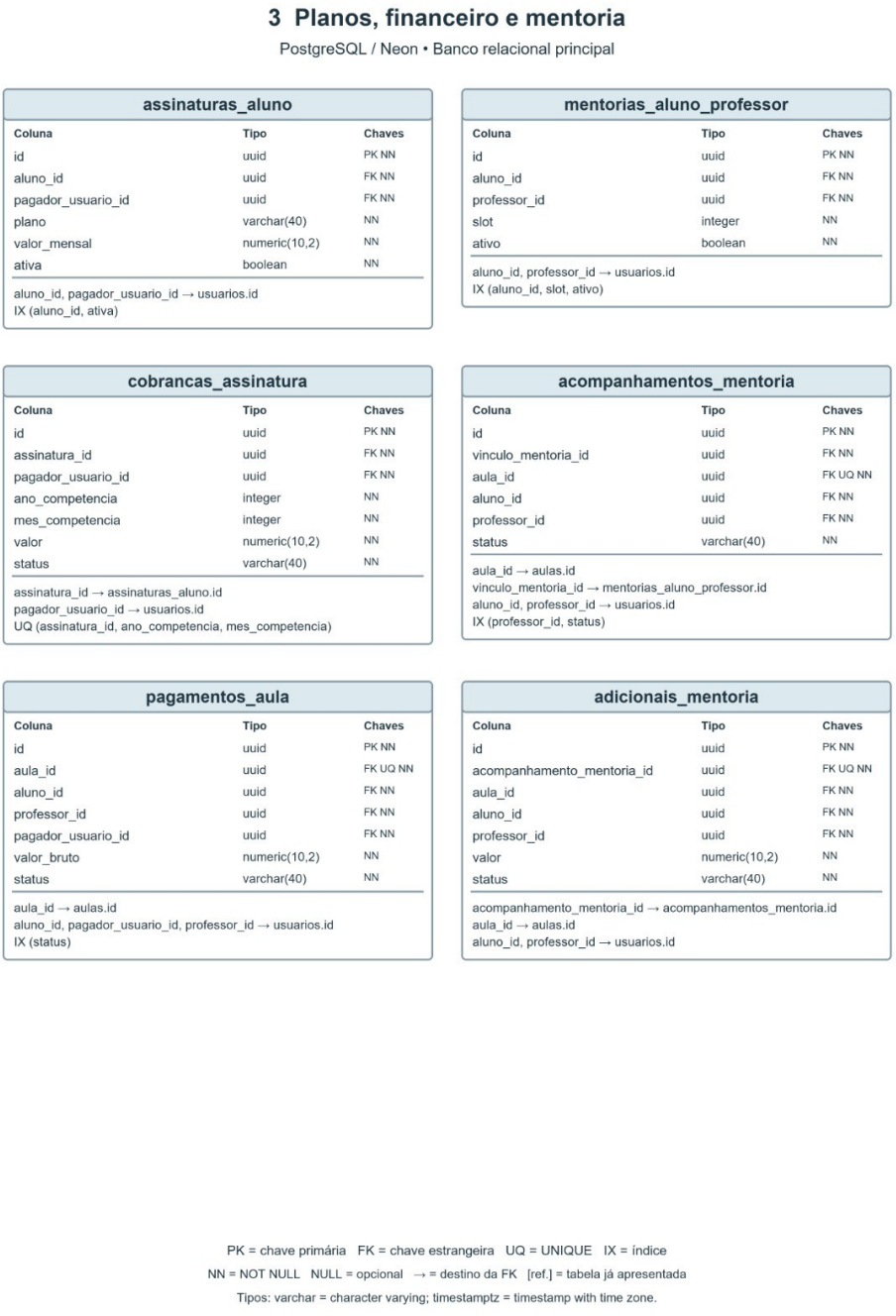

Fonte: Elaborado pelos autores (2026).

**Figura 18 – Modelo físico (2/6): aulas e recursos pedagógicos**



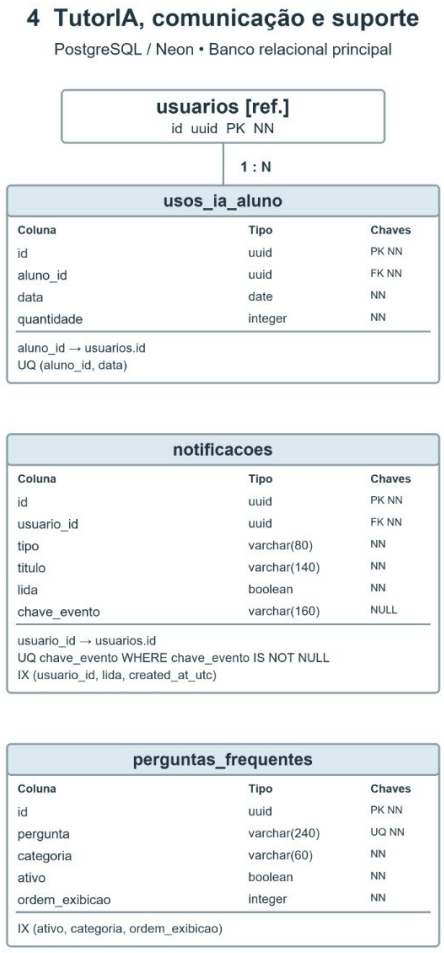

Fonte: Elaborado pelos autores (2026).

**Figura 19 – Modelo físico (3/6): planos, assinaturas, financeiro e Mentoria**



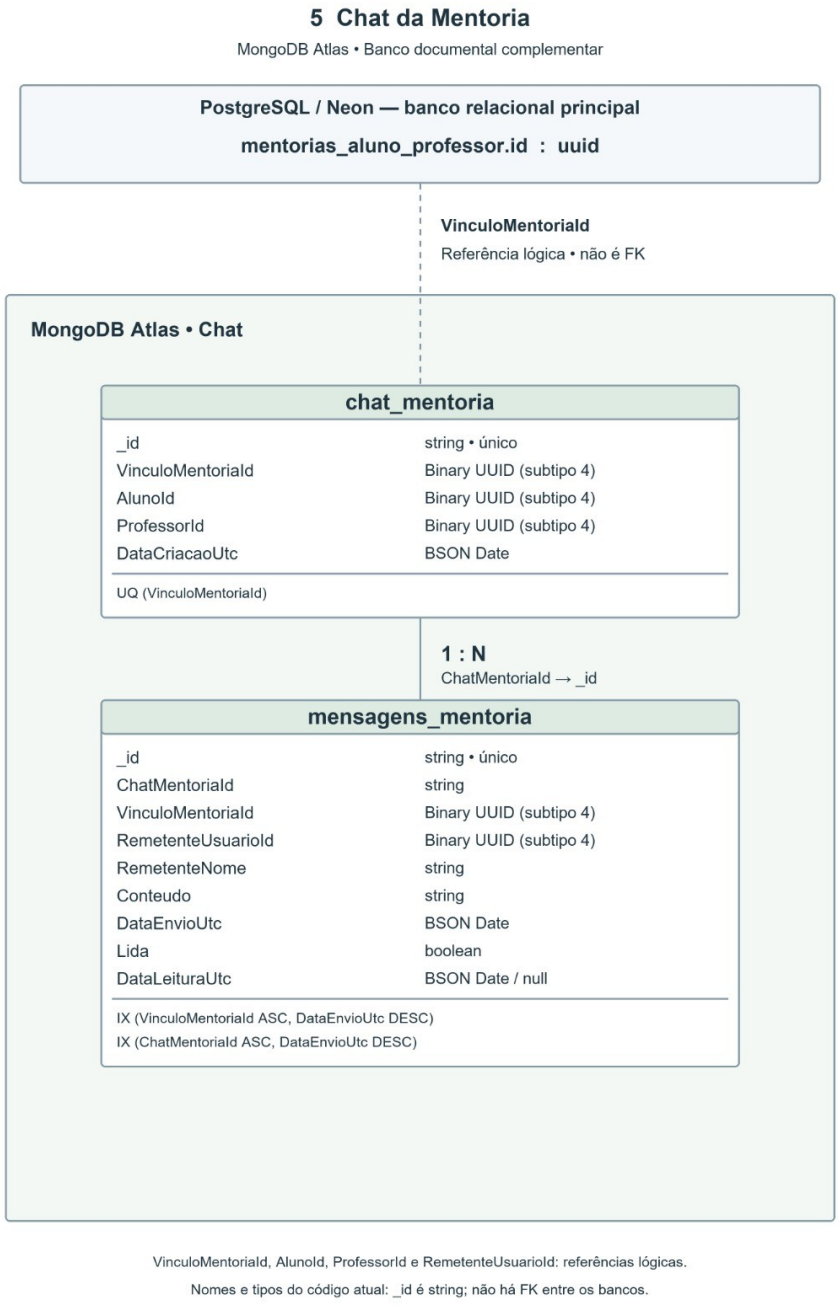

Fonte: Elaborado pelos autores (2026).

**Figura 20 – Modelo físico (4/6): TutorIA, comunicação e suporte**



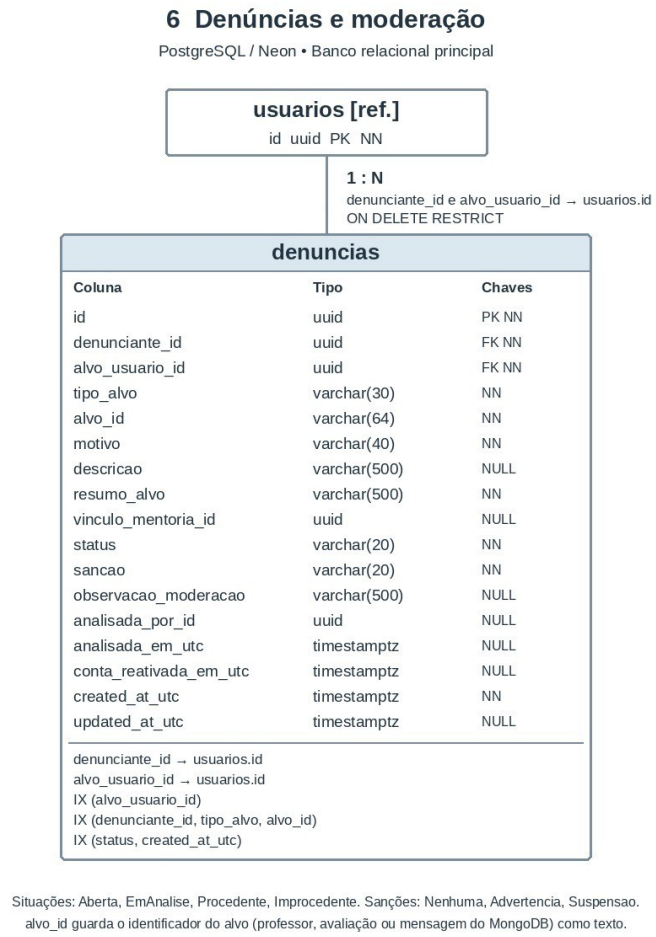

Fonte: Elaborado pelos autores (2026).

**Figura 21 – Modelo físico (5/6): chat da Mentoria (MongoDB Atlas)**




Fonte: Elaborado pelos autores (2026).

**Figura 22 – Modelo físico (6/6): denúncias e moderação**




Fonte: Elaborado pelos autores (2026).